



卷积神经网络-1

Convolutional Neural Network (CNN)-1



01 引言 (Introduction)

02 基础组件(Basic Components)

03 卷积网络搭建(CNN Construction)

04 小结与资源 (Take Home Message and Resource)

CNN

PART ONE

引言

Introduction

■ 全连接前馈神经网络

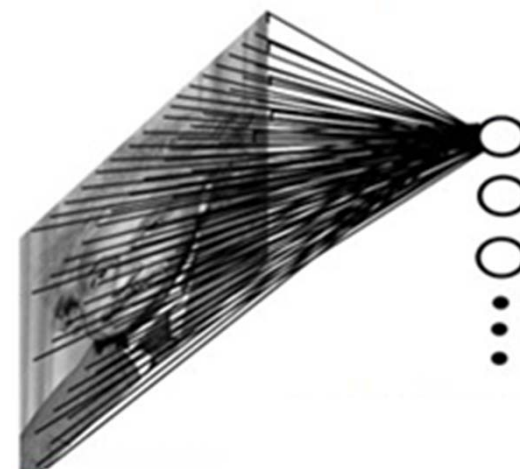
➤ (1) 权重矩阵的参数过多

- **思考：**对于大小为 1000×1000 像素的输入图像，假设第一个全连接层有1000个节点，问该层的参数量是多少？

输入层节点数： $1000 \times 1000 = 100$ 万

第一层参数量： $(1000 \times 1000 + 1) \times 1000 = 10$ 亿！

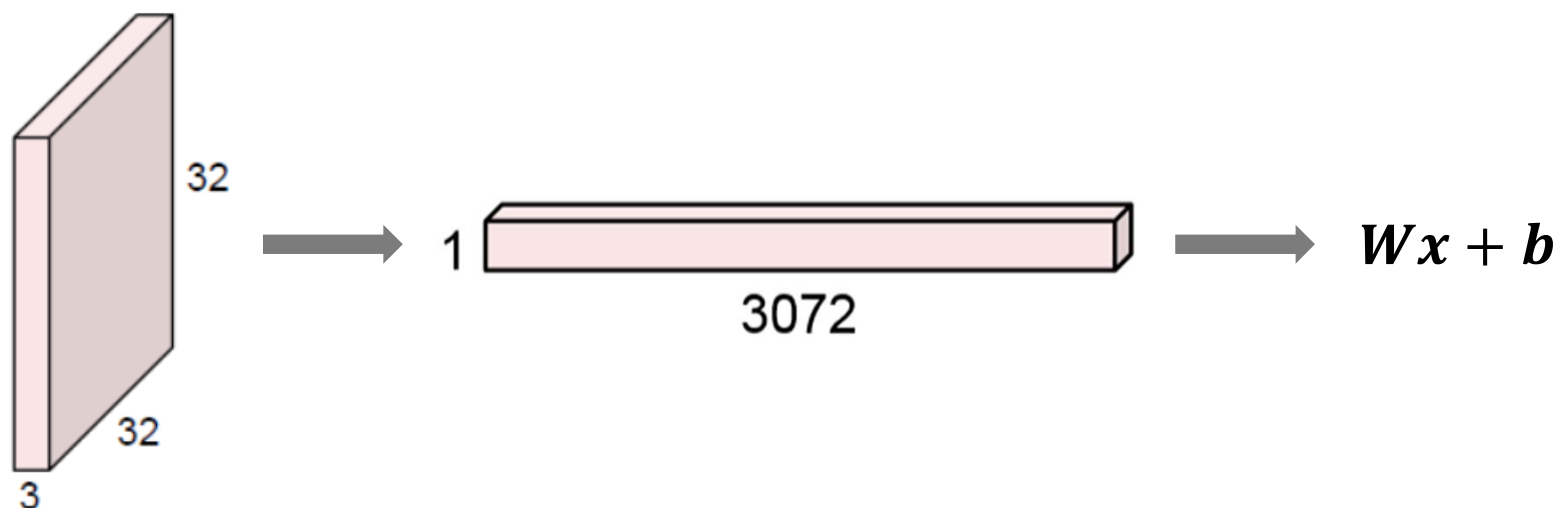
- 参数量会随着输入图像尺寸的增大而激增
- 对图像处理的延展性差



■ 全连接前馈神经网络

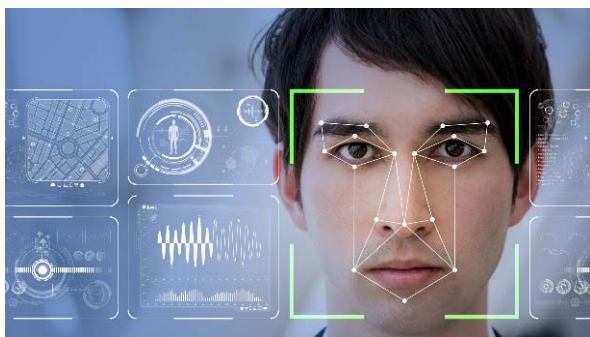
➤ (2) 忽略了输入数据的局部不变性特征

- 自然图像中的物体具有局部不变性特征
 - 尺度缩放、平移、旋转等操作不影响其语义信息
- 全连接网络很难提取这些局部不变特征



■ 卷积神经网络 (Convolutional Neural Networks, CNN)

- 专门为具有类似网状 (grid-like) 结构的数据而生 (LeCun et al. 98)
 - 1-D grid: 时间序列化数据
 - 2-D grid: 图像数据
 - 3-D grid: 视频数据
- 广泛应用于多种计算机视觉任务, 如人脸识别、视觉跟踪、目标检测...

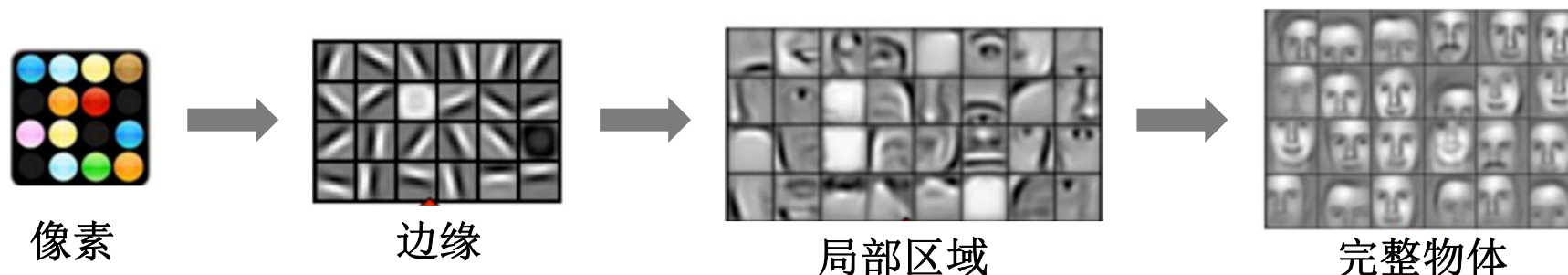


■ CNN的重要里程碑

- 1962年, Hubel和Wiesel在研究猫脑视觉皮层系统时, 提出**感受野 (Receptive Field)** 的概念, 且发现视觉皮层对信息的分层处理机制, 获得了诺贝尔生理学或医学奖。

在视觉神经系统中, 一个神经元的感受野指视网膜上的特定区域, 只有这个区域内的刺激才能够激活该神经元。

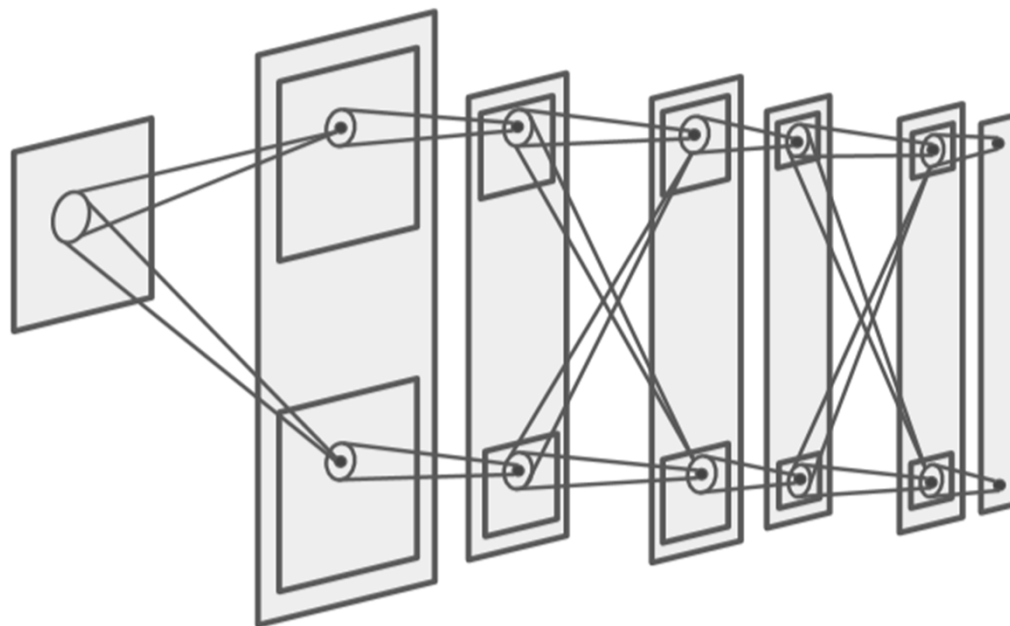
- 视觉信息从视网膜传递到大脑中是通过多个层次的感受野激发完成



Hubel DH, Wiesel TN. Receptive fields, binocular interaction and functional architecture in the cat's visual cortex. Journal of Physiology. 1962;160(1):106-154.

■ CNN的重要里程碑

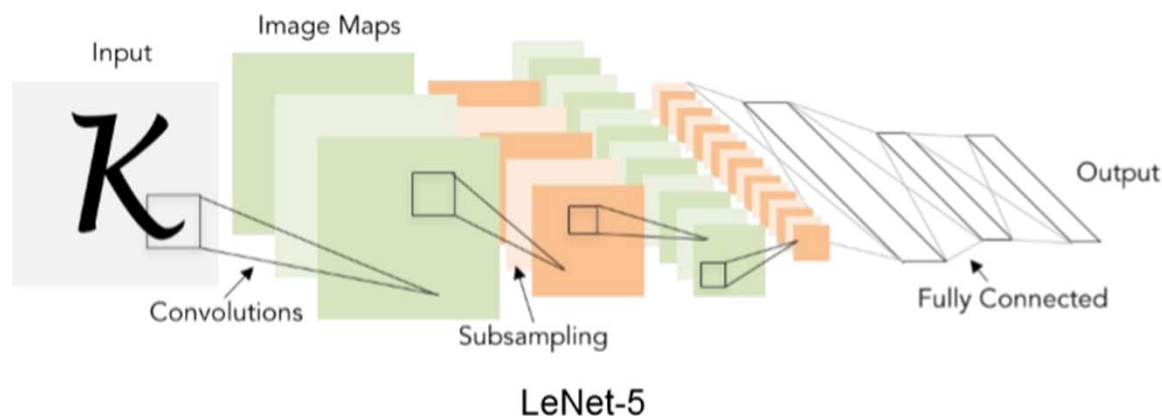
- 1980年, Fukushima等基于感受野提出的神经感知机 (Neocognitron), 可视为CNN的第一次实现, 也是第一个基于神经元之间的局部连接性和层次结构组织的人工神经网络。



Fukushima K, Miyake S. Neocognitron: A new algorithm for pattern recognition tolerant of deformations and shifts in position. Pattern Recognition. 1982;15(6):455-69.

■ CNN的重要里程碑

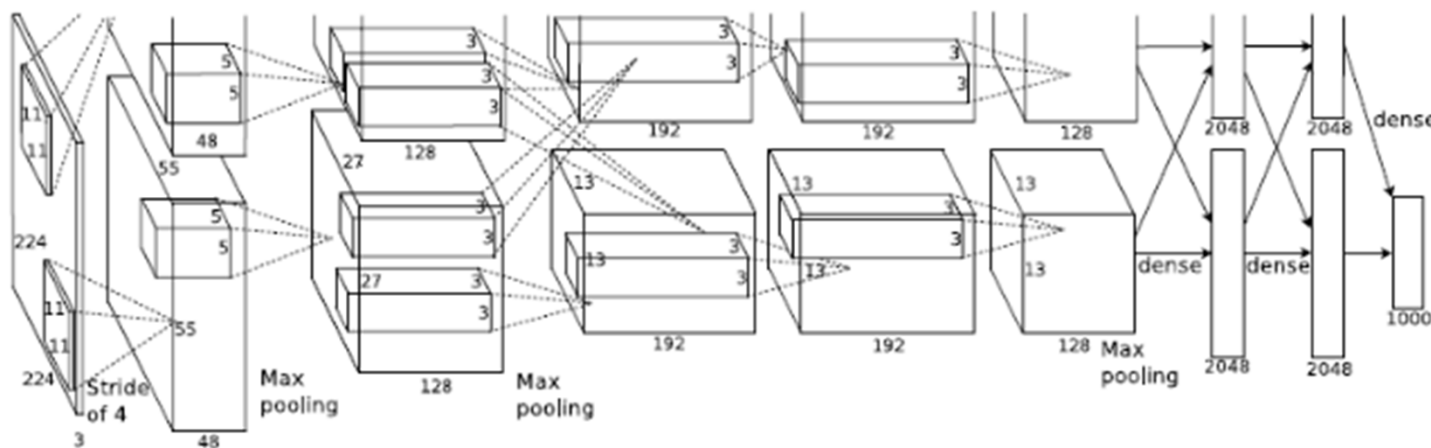
- 1990年，Lecun等在研究手写数字识别问题时，首先提出了使用梯度反向传播算法训练的CNN模型LeNet-5，在MNIST手写数字数据上表现出色。



LeCun Y, Boser B, Denker J, Henderson D, Howard R. Handwritten digit recognition with a backpropagation network. World of Computer Science & Information Technology Journal. 1990;2:299-304.

■ CNN的重要里程碑

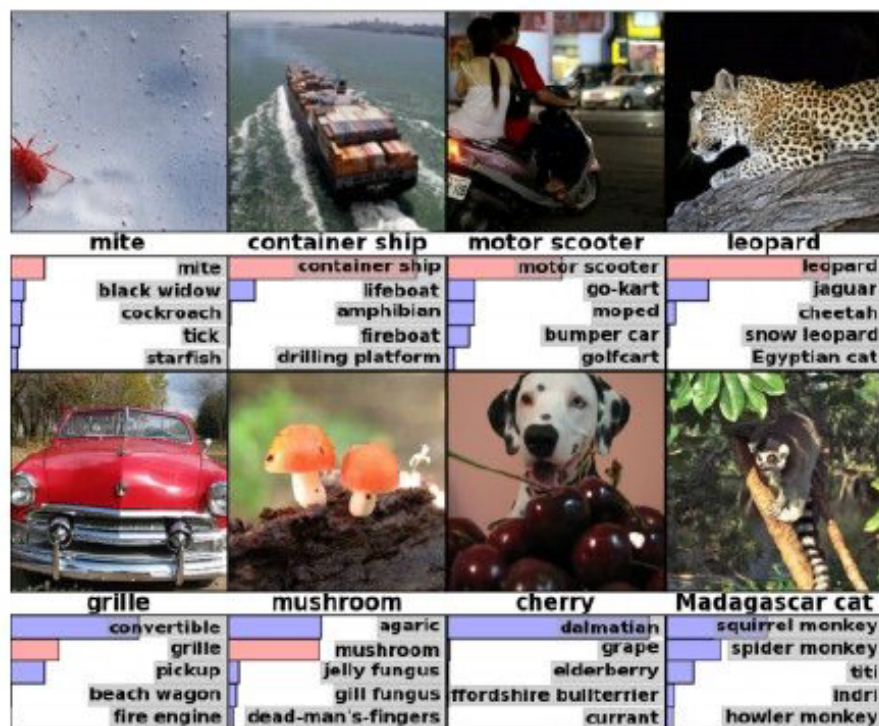
- 直至2012年，在Imagenet图像识别大赛中，Hinton等人基于CNN提出的AlexNet大放异彩，并引领了CNN的复兴，此后CNN的研究进入高速发展。



A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet Classification with Deep Convolutional Neural Networks. in Advances in neural information processing systems, 2012, pp. 1097–1105.

■ 至今，CNN无处不在

图像分类



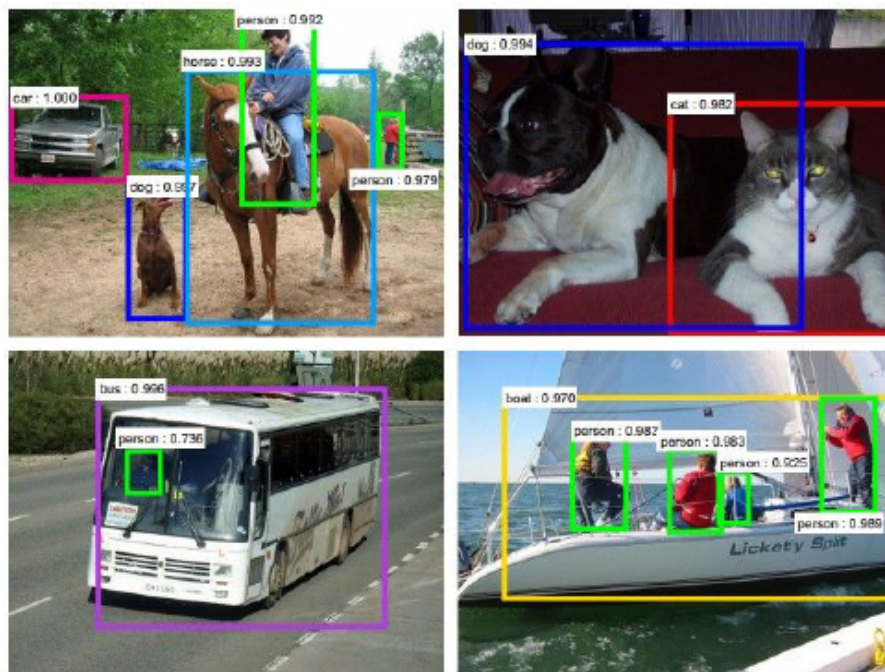
图像检索



Figures copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

■ 至今，CNN无处不在

目标检测



Figures copyright Shaoqing Ren, Kaiming He, Ross Girshick, Jian Sun, 2015. Reproduced with permission.

[Faster R-CNN: Ren, He, Girshick, Sun 2015]

语义分割



Figures copyright Clement Farabet, 2012. Reproduced with permission.

[Farabet et al., 2012]

- 至今，CNN无处不在

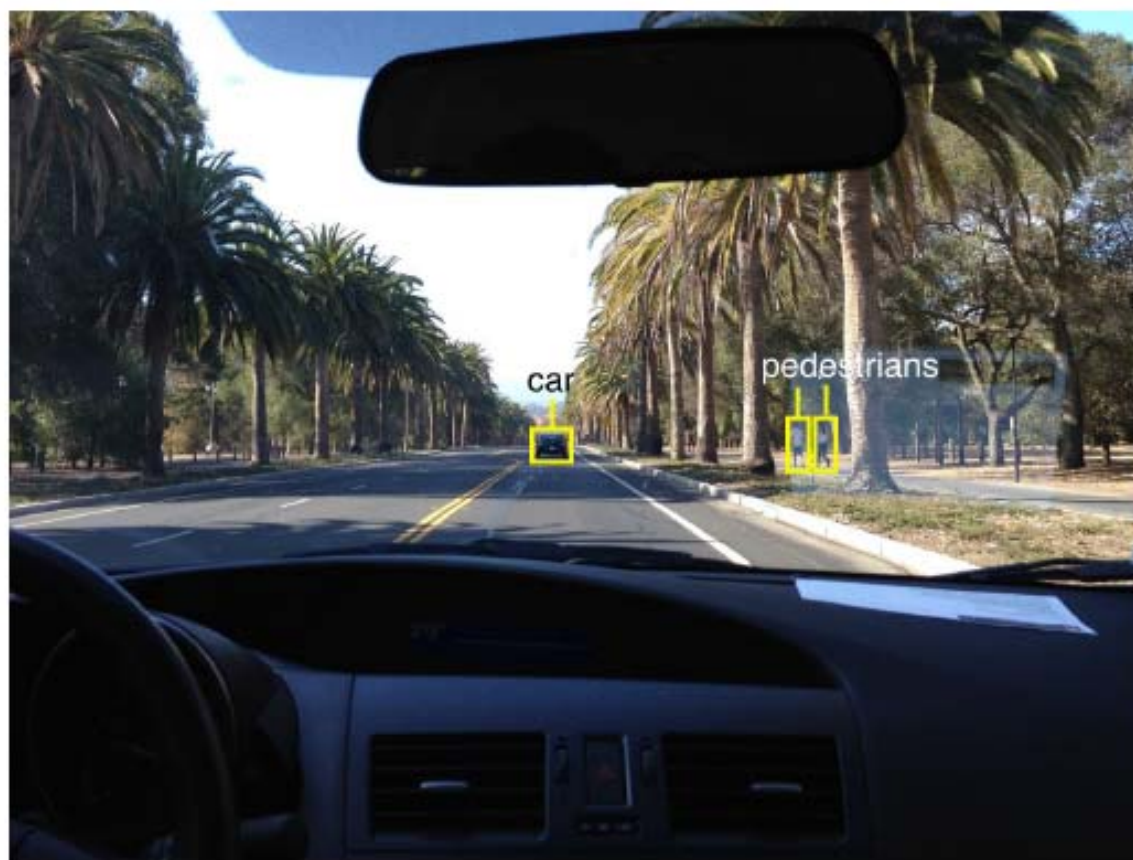


Photo by Lane McIntosh. Copyright CS231n 2017.

自动驾驶

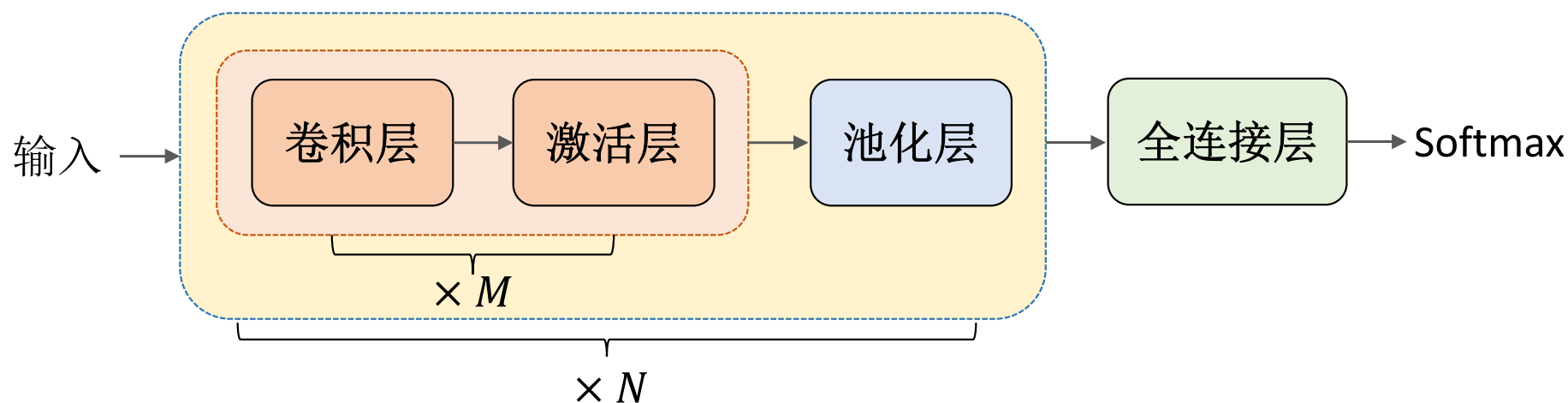
PART TWO

基础组件

Basic Components

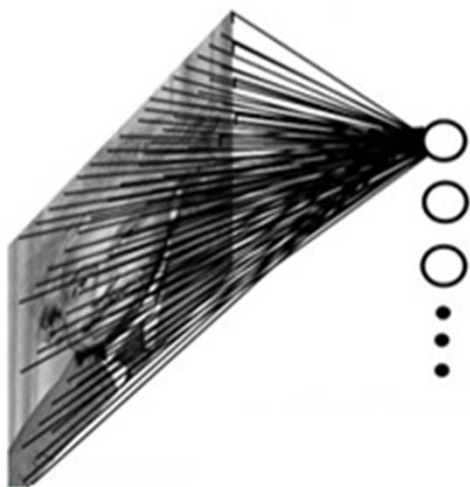
■ CNN概述

- 一种包含卷积计算的前馈神经网络
- 通常含有多个卷积层 (Convolutional Layer)、激活层 (Activation Layer) 和池化层 (Pooling Layer)
- 重要属性：局部连接、权值共享、平移不变性

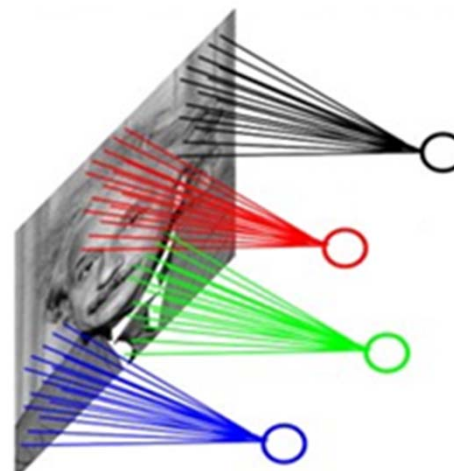


■ CNN概述

- 局部连接：每个神经元不再和上一层的所有神经元相连，而只和一小部分神经元相连，从而可显著降低网络参数量。



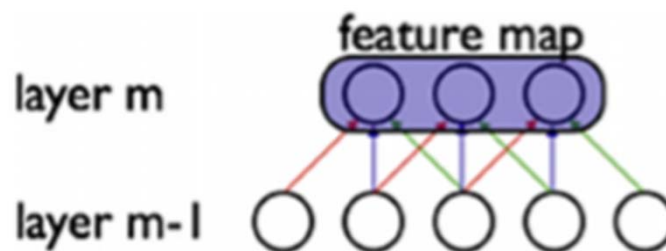
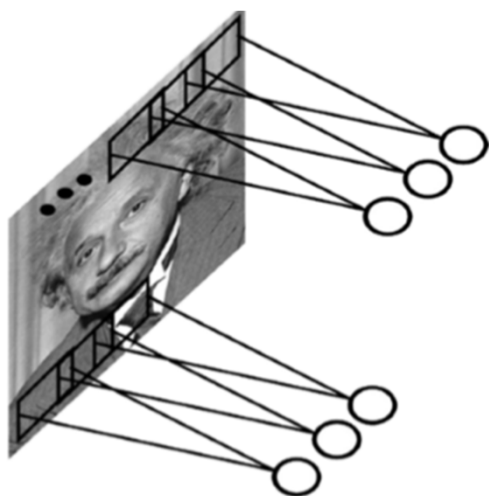
全连接层



卷积层

■ CNN概述

- 权值共享：不同位置处的神经元共享同一组权重（**滤波器**），可进一步降低网络参数量

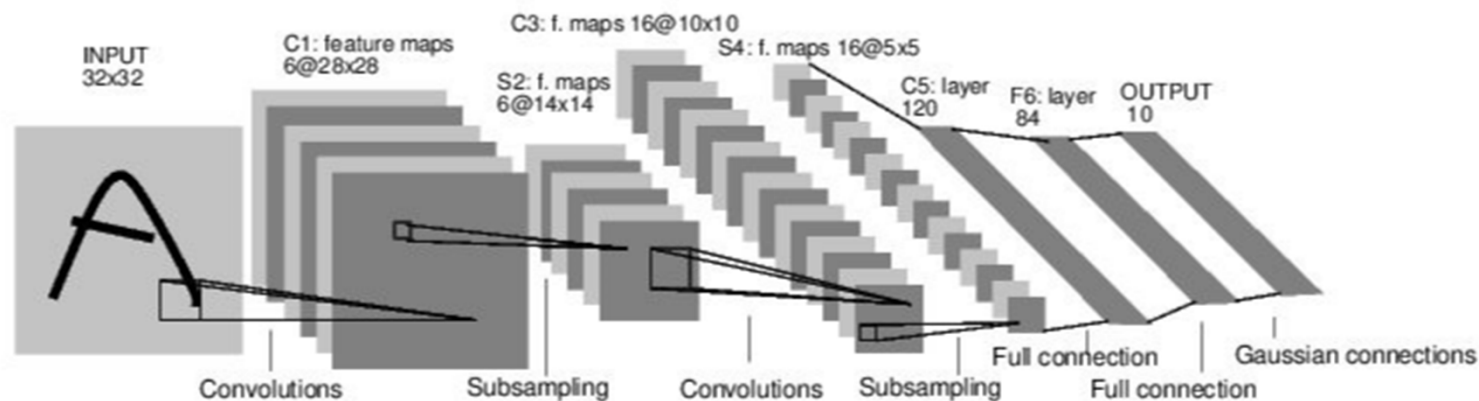


相同颜色的权重值相同

- 平移不变性：滤波器可捕获与图像空间位置无关的区域特征
 - 图像中不同位置可能出现相似的图像特性（边缘、纹理、类别）

■ 三种基础组件

- 卷积层 (Convolutional Layer)
- 激活层 (Activation Layer)
- 池化层 (Pooling Layer)



■ 卷积

- 卷积经常用在信号处理中，用于计算信号的延迟累积
- 假设信号发生器每个时刻 t 产生一个信号 x_t ，其信息衰减率为 w_k ，即在 $(k-1)$ 个时间步长后，信息变为原来的 w_k 倍
 - 假设 $w_1 = 1, w_2 = \frac{1}{2}, w_3 = 1/4$
- 时刻 t 收到的信号 y_t 为当前时刻产生的信息和以前时刻延迟信息的叠加：

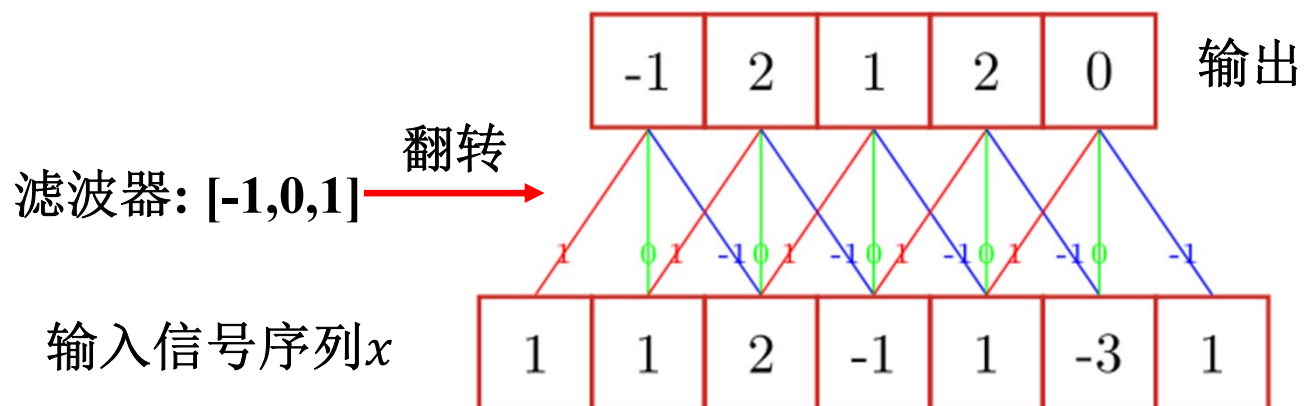
$$\begin{aligned}y_t &= 1 \times x_t + 1/2 \times x_{t-1} + 1/4 \times x_{t-2} \\&= w_1 \times x_t + w_2 \times x_{t-1} + w_3 \times x_{t-2} \\&= \sum_{k=1}^3 w_k \cdot x_{t-k+1}.\end{aligned}$$

↓
滤波器 (filter) 或卷积核 (convolution kernel)

■ 卷积

➤ 给定一个输入信号序列 x 和滤波器 w ,卷积的输出为:

$$y_t = \sum_{k=1}^K w_k x_{t-k+1}$$

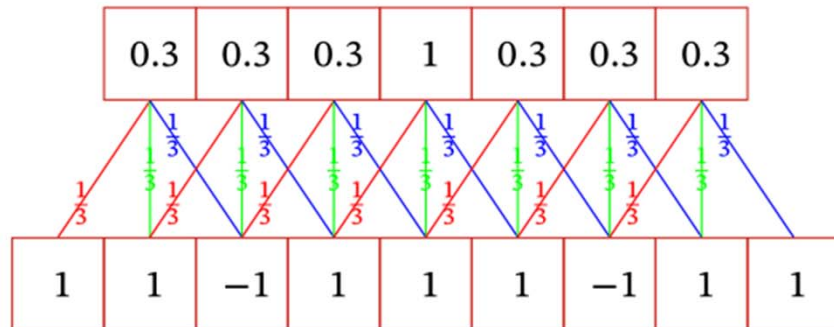


■ 卷积

➤ 不同的滤波器来提取信号序列中的不同特征

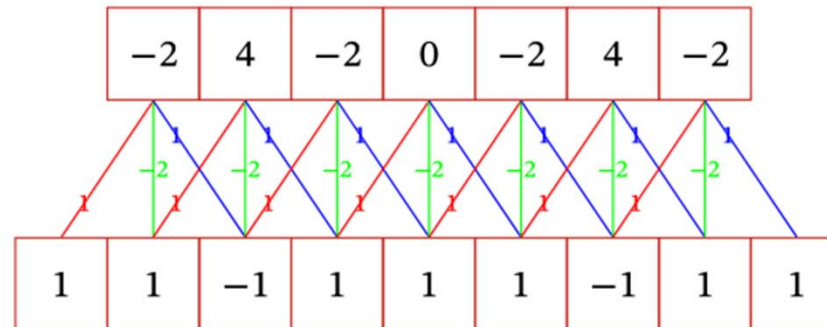
滤波器 $\mathbf{w} = [1/K, \dots, 1/K]$ 时，卷积相当于信号序列的简单移动平均（窗口大小为 K ）

当令滤波器 $\mathbf{w} = [1, -2, 1]$ 时，可以近似实现对信号序列的二阶微分，即 $x''(t) = x(t+1) + x(t-1) - 2x(t)$



(a) 滤波器 $[1/3, 1/3, 1/3]$

低频信息



(b) 滤波器 $[1, -2, 1]$

高频信息

■ 二维卷积

➤ 在图像处理中，图像是以二维矩阵的形式输入到神经网络中，因此我们需要二维卷积。

- 一个输入信息 X 与滤波器 W 的二维卷积定义为：

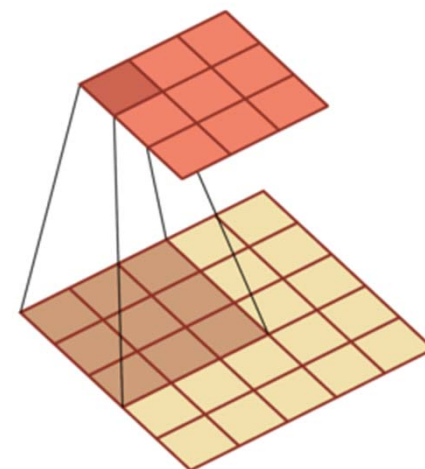
$$Y = W * X$$

$$y_{ij} = \sum_{u=1}^U \sum_{v=1}^V w_{uv} x_{i-u+1, j-v+1}$$

1	1	1	1	1
-1	0	-3	0	1
2	1	1	-1	0
0	-1	1	2	1
1	2	1	1	1

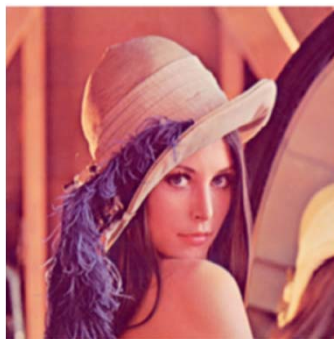
$\times -1$ $\times 0$ $\times 0$
 $\times 0$ $\times 0$ $\times 0$
 $\times 0$ $\times 0$ $\times 1$

$$* \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 0 & -2 & -1 \\ 2 & 2 & 4 \\ -1 & 0 & 0 \end{bmatrix}$$



■ 二维卷积

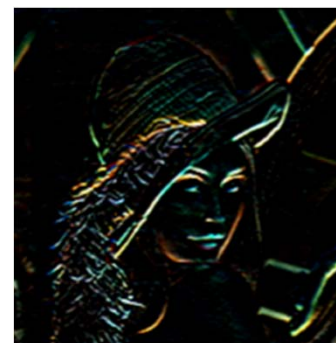
➤ 卷积作为特征提取器



*

1	2	1
0	0	0
-1	-2	-1

=



水平边缘

1	0	-1
2	0	-2
1	0	-1

=



垂直边缘

■ 互相关

- 信号处理中，计算卷积需要对滤波器 (卷积核) 进行**翻转**（**翻转**指从两个维度（从上到下、从左到右）颠倒次序，即旋转**180度**）
- 在卷积神经网络中，卷积操作的目标是**提取特征**；滤波器参数并非人为设定，而是通过网络训练学习得到的

翻转是不必要的！

- **互相关**:是一个衡量两个序列相关性的函数，通常是用滑动窗口的点积计算来实现

$$y_{ij} = \sum_{u=1}^m \sum_{v=1}^n w_{uv} \cdot x_{i+u-1, j+v-1}$$

在神经网络中使用卷积是为了进行特征抽取，卷积核是否进行翻转和其特征抽取的能力无关。特别是当卷积核是可学习的参数时，卷积和互相关在能力上是等价的

除非特别声明，CNN中的卷积一般指“互相关”

■ 卷积运算

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6×6的灰度图像

*

1	0	-1
1	0	-1
1	0	-1

3×3的滤波器

=

■ 卷积运算

3 ¹	0 ⁰	1 ⁻¹	2	7	4
1 ¹	5 ⁰	8 ⁻¹	9	3	1
2 ¹	7 ⁰	2 ⁻¹	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6×6的灰度图像

*

1	0	-1
1	0	-1
1	0	-1

3×3的滤波器

=

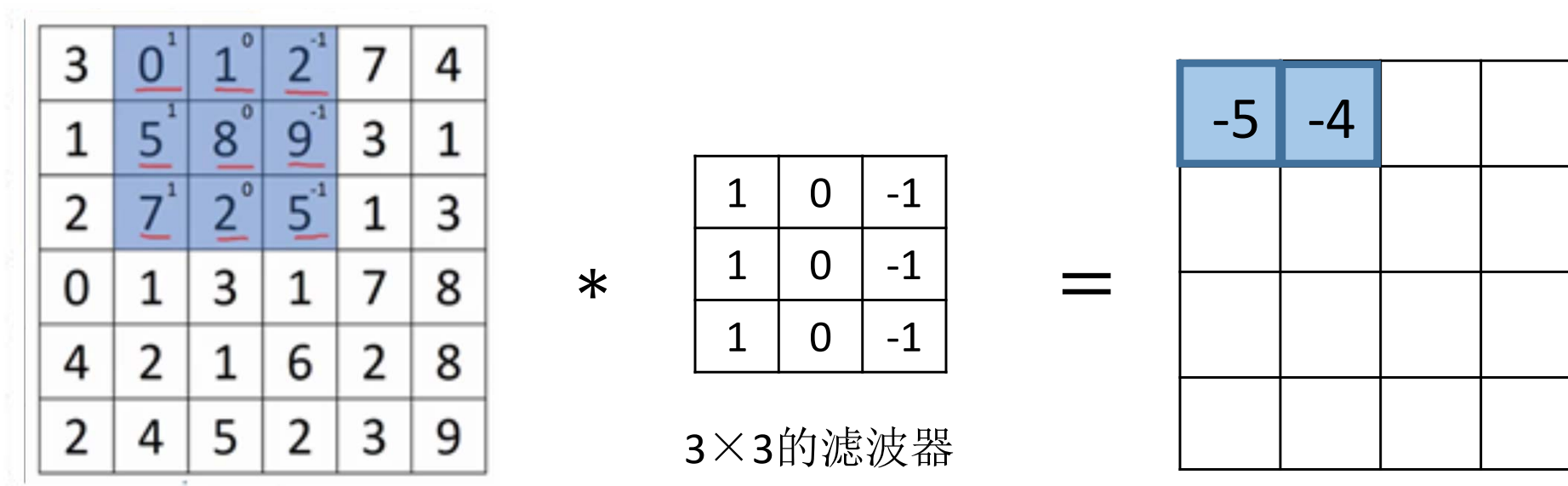
-5			

“*”：对应点元素相乘，然后再将该矩阵每个元素相加

$$\begin{bmatrix} 3 \times 1 & 0 \times 0 & 1 \times (-1) \\ 1 \times 1 & 5 \times 0 & 8 \times (-1) \\ 2 \times 1 & 7 \times 0 & 2 \times (-1) \end{bmatrix} = \begin{bmatrix} 3 & 0 & -1 \\ 1 & 0 & -8 \\ 2 & 0 & -2 \end{bmatrix}$$

$$3 + 1 + 2 + 0 + 0 + 0 + (-1) + (-8) + (-2) = -5$$

■ 卷积运算



6×6 的灰度图像

3×3 的滤波器

“*”：对应点元素相乘，然后再将该矩阵每个元素相加

继续做同样的元素乘法，然后加起来：

$$0 \times 1 + 5 \times 1 + 7 \times 1 + 1 \times 0 + 8 \times 0 + 2 \times 0 + 2 \times (-1) + 9 \times (-1) + 5 \times (-1) = -4$$

■ 卷积运算

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6×6的灰度图像

*

1	0	-1
1	0	-1
1	0	-1

3×3的滤波器

=

-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2	-3	-16

4×4的图像

“*”：对应点元素相乘，然后再将该矩阵每个元素相加

继续做同样的元素乘法，然后加起来，直到遍历了整个矩阵

■ 边缘检测

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

6×6的灰度图像

*

1	0	-1
1	0	-1
1	0	-1

3×3的过滤器

=

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0

4×4的图像



*



- 在CNN中，滤波器的参数通过网络学习而获得，并应用于整幅图像中进行图像的特征提取

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

$W \in \mathcal{R}^{3 \times 3}$

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

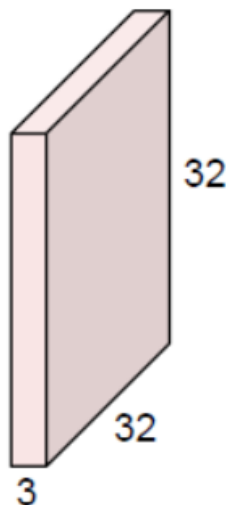
=

- 图像经过卷积后得到特征图 (Feature Map)
- 卷积核看成一个特征提取器

■ 典型的卷积层

- 通常输入图像为三维数据，即**RGB**图像

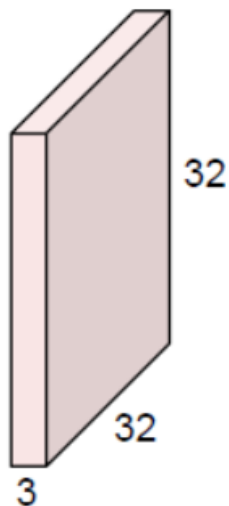
$32 \times 32 \times 3$ 图像



■ 典型的卷积层

- 通常输入图像为三维数据，即**RGB**图像

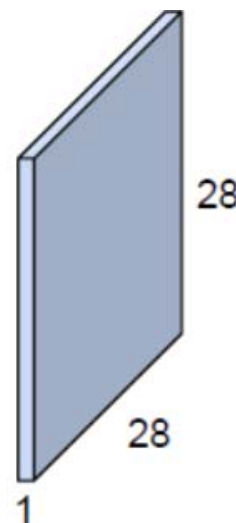
$32 \times 32 \times 3$ 图像



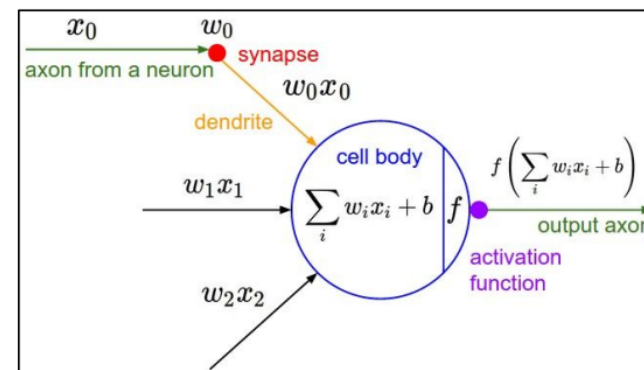
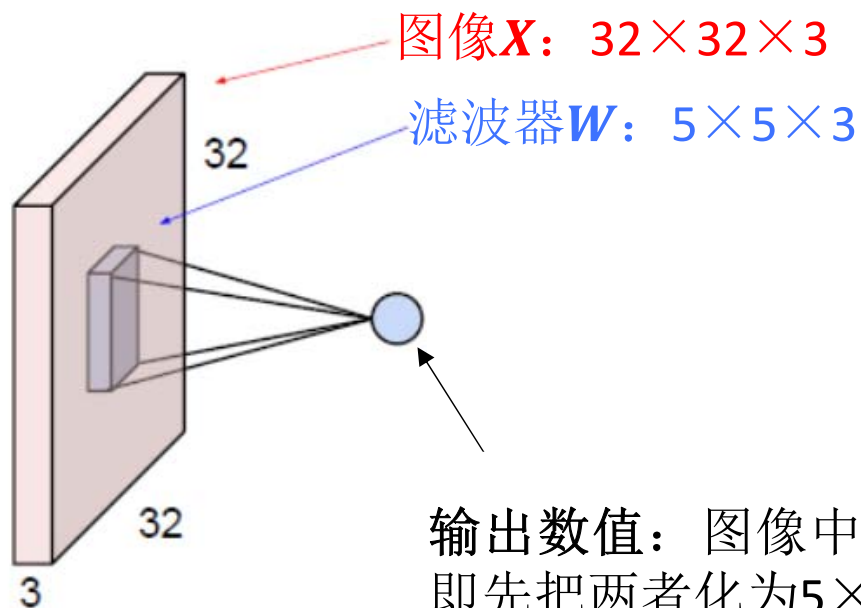
$5 \times 5 \times 3$ 滤波器



滤波器与图像做卷积
点乘操作滑遍整张图像

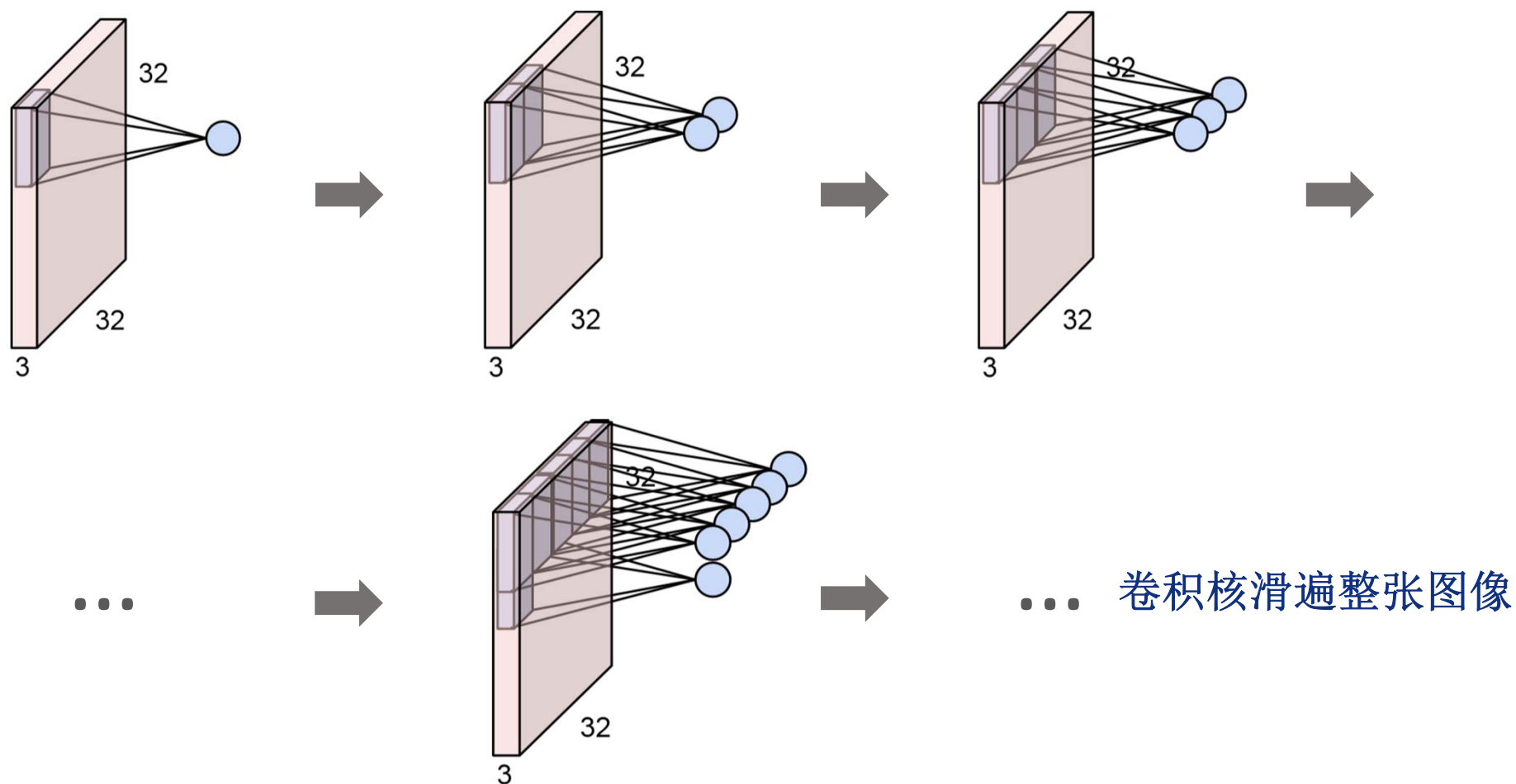


■ 典型的卷积层



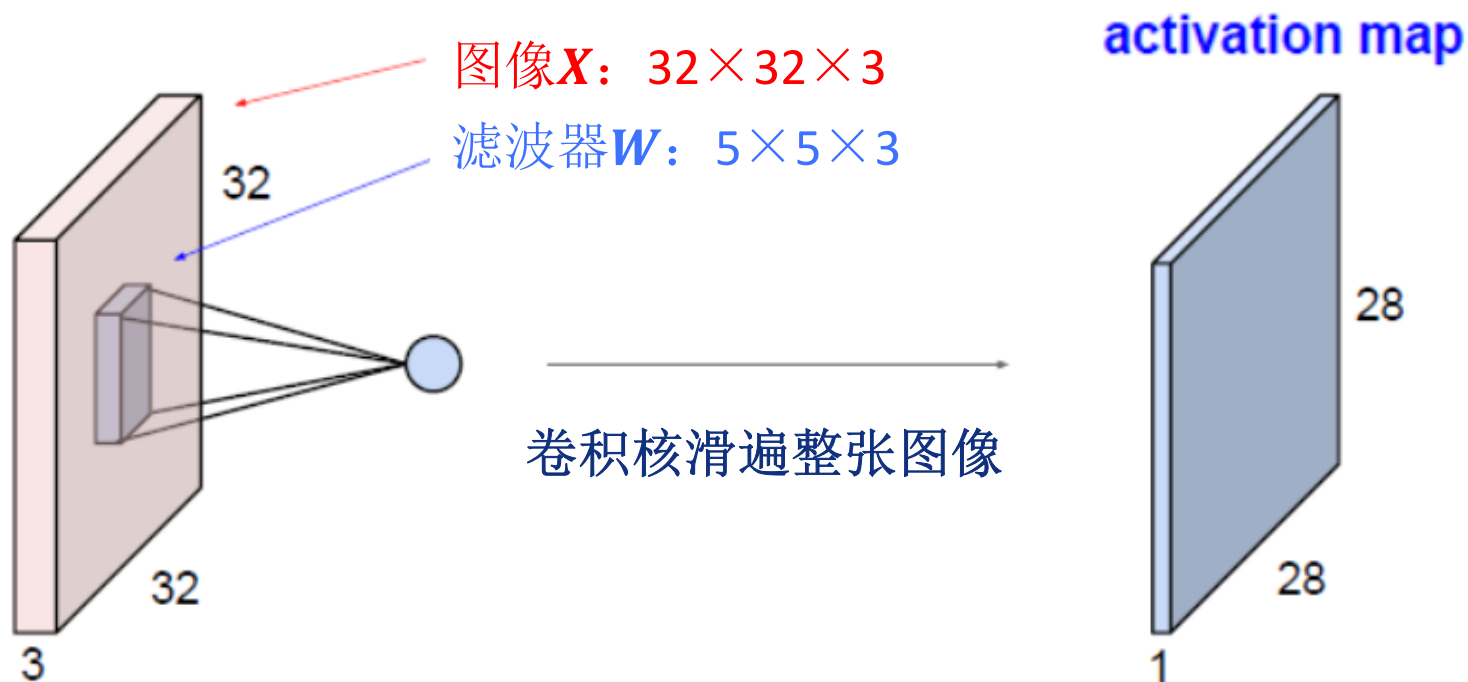
即局部连接的神经元

■ 典型的卷积层



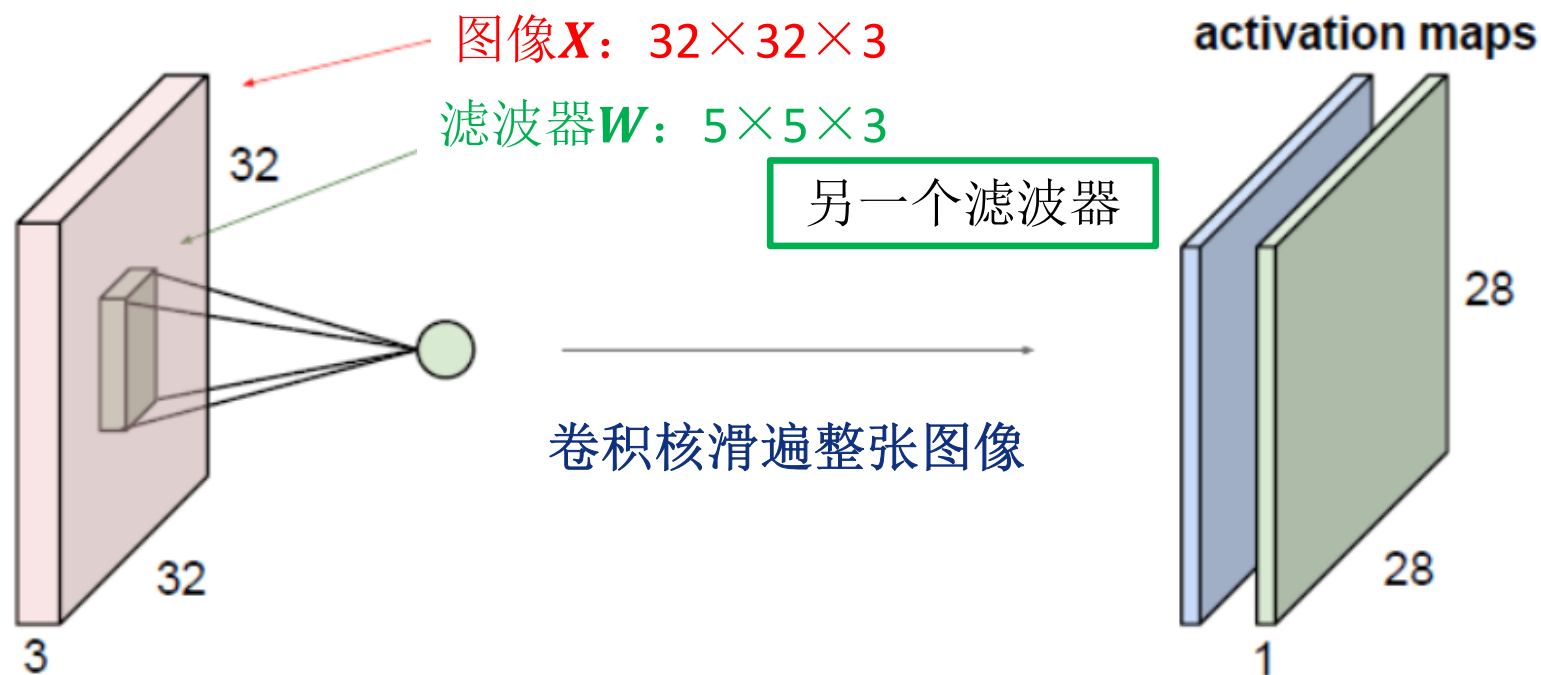
■ 典型的卷积层

➤ 单个滤波器



■ 典型的卷积层

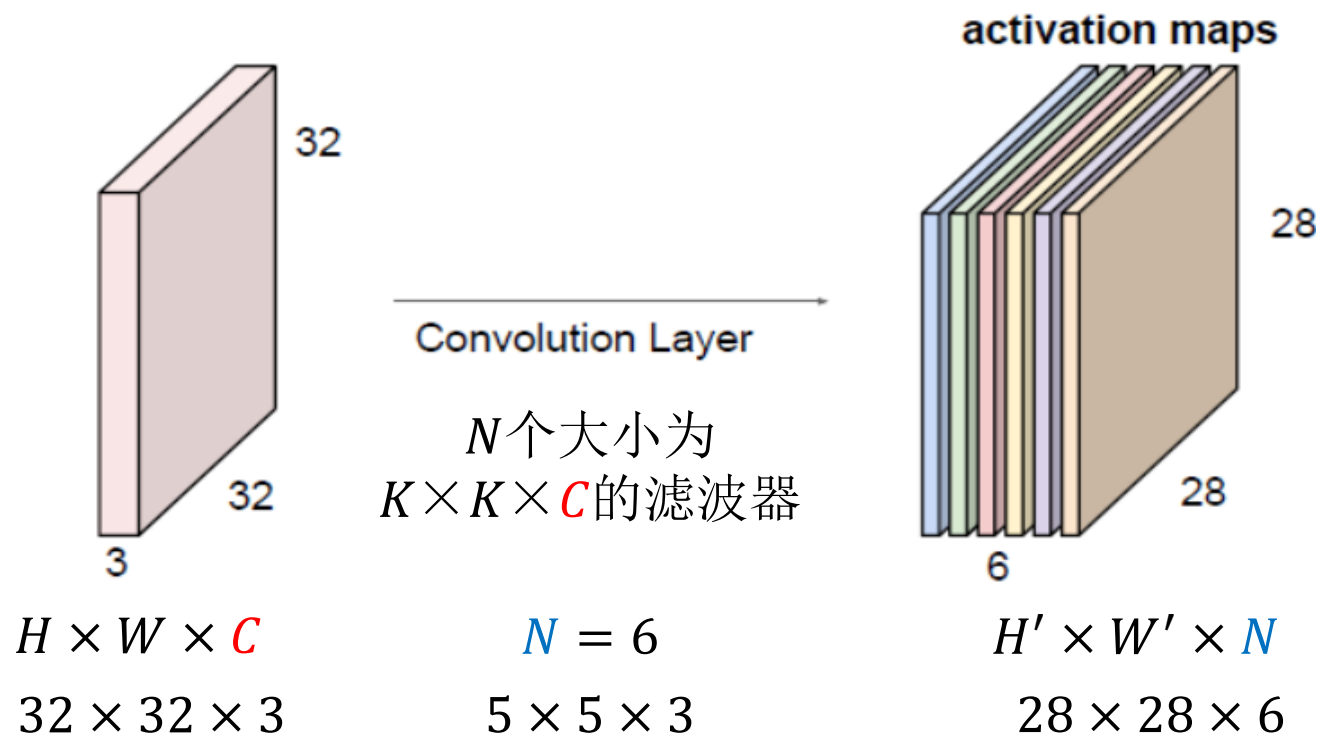
➤ 多个滤波器



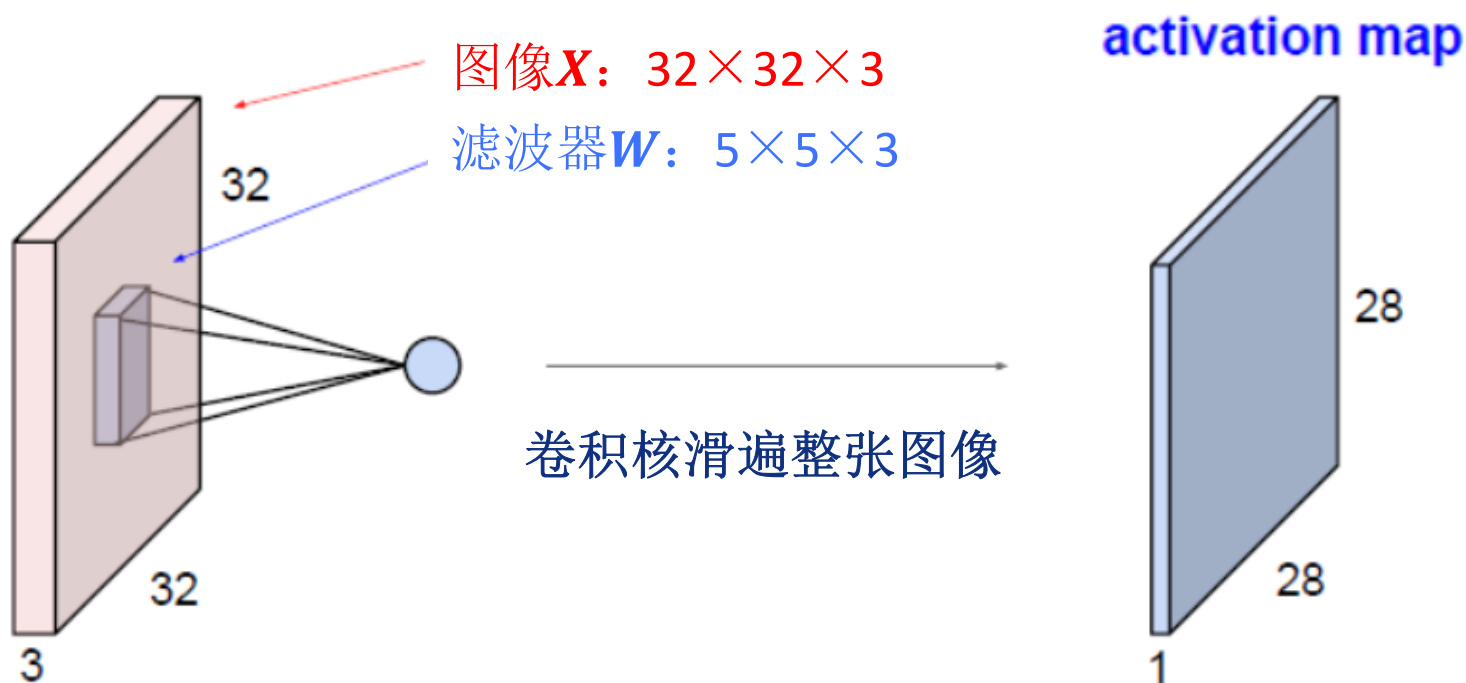
■ 典型的卷积层

➤ 多个滤波器

- 若有6个 $5 \times 5 \times 3$ 的滤波器，则得到6个不同的响应图 (Activation Map)
- 6个activation maps堆叠起来，构成一张大小为 $28 \times 28 \times 6$ “新图像”，称为特征图 (Feature Map)

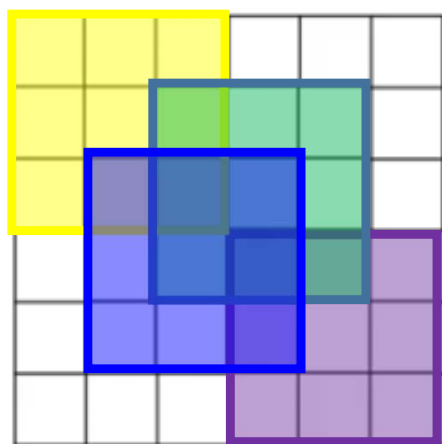


➤ 进一步观察卷积过程的**空间维度**



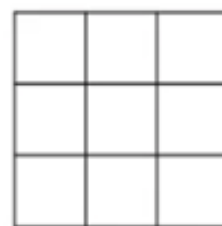
➤ 进一步观察卷积过程的空间维度

- 每次做卷积操作，图像就会缩小，随着卷积次数的增加图像会越来越小
- 位于角落或者边缘区域的像素点在输出中采用较少，意味着丢掉了图像边缘位置的许多信息



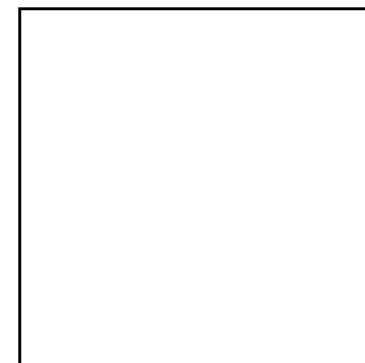
6×6
 $h \times w$

*



3×3
 $k \times k$

=

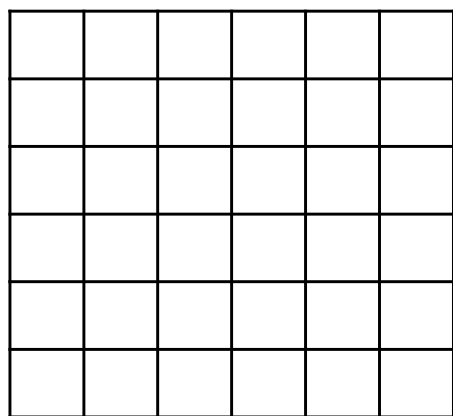


4×4
 $(h-k+1) \times (w-k+1)$

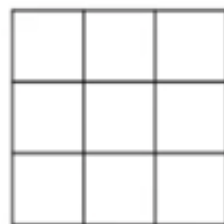
空间
维度

■ 填充 (Padding)

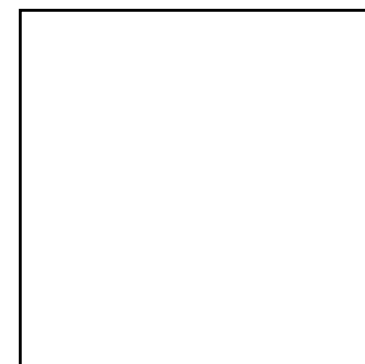
- 填充是构建卷积层的一个基本操作
- 在空间维度上，对卷积层输入的四周各填充 p 个像素



*



=



空间
维度

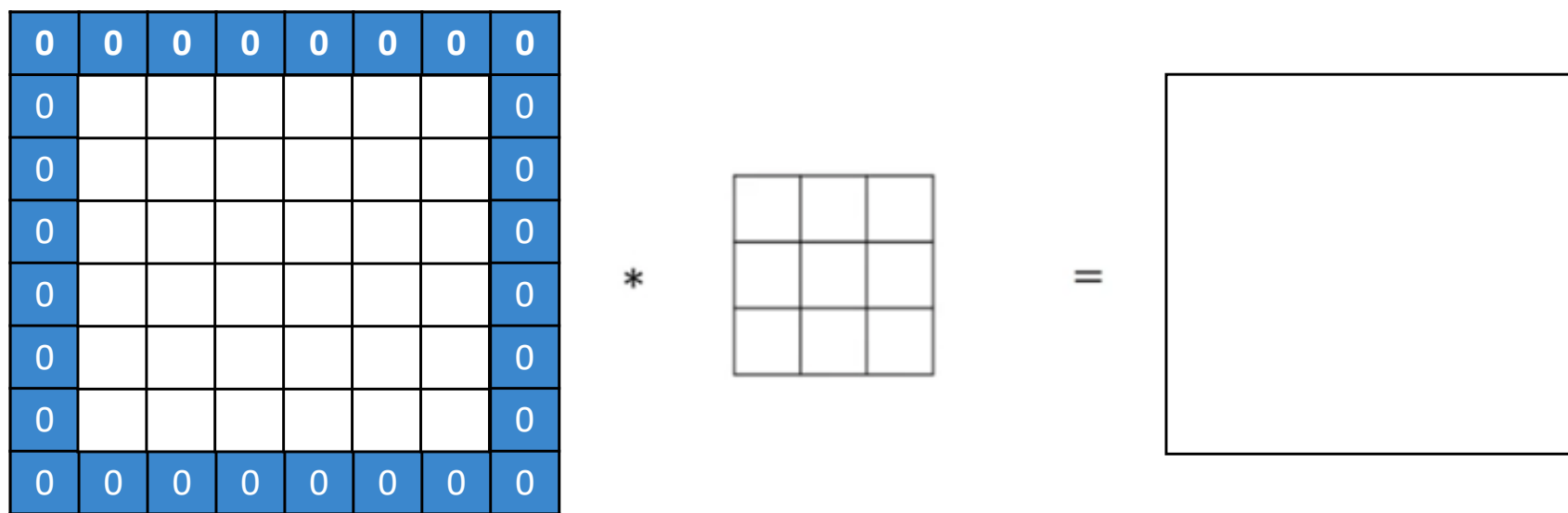
6×6

3×3

4×4

■ 填充 (Padding)

- 填充是构建卷积层的一个基本操作
- 在空间维度上，对卷积层输入的四周各填充 p 个像素



空间
维度

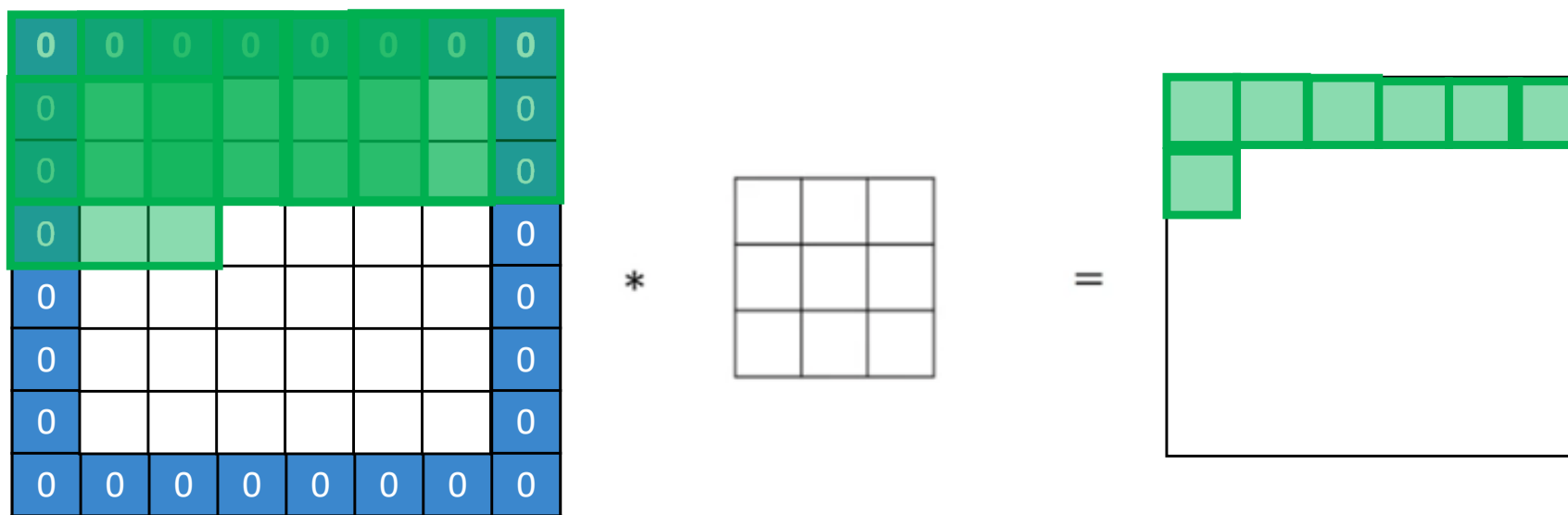
$$\begin{array}{ccc} 6 \times 6 & \xrightarrow{p=1} & 8 \times 8 \\ h \times w & (h+2p) \times (w+2p) & \end{array}$$

$$\begin{array}{ccc} 3 \times 3 \\ k \times k & \end{array}$$

$$4 \times 4 \longrightarrow ? \times ?$$

■ 填充 (Padding)

- 填充是构建卷积层的一个基本操作
- 在空间维度上，对卷积层输入的四周各填充 p 个像素



空间
维度

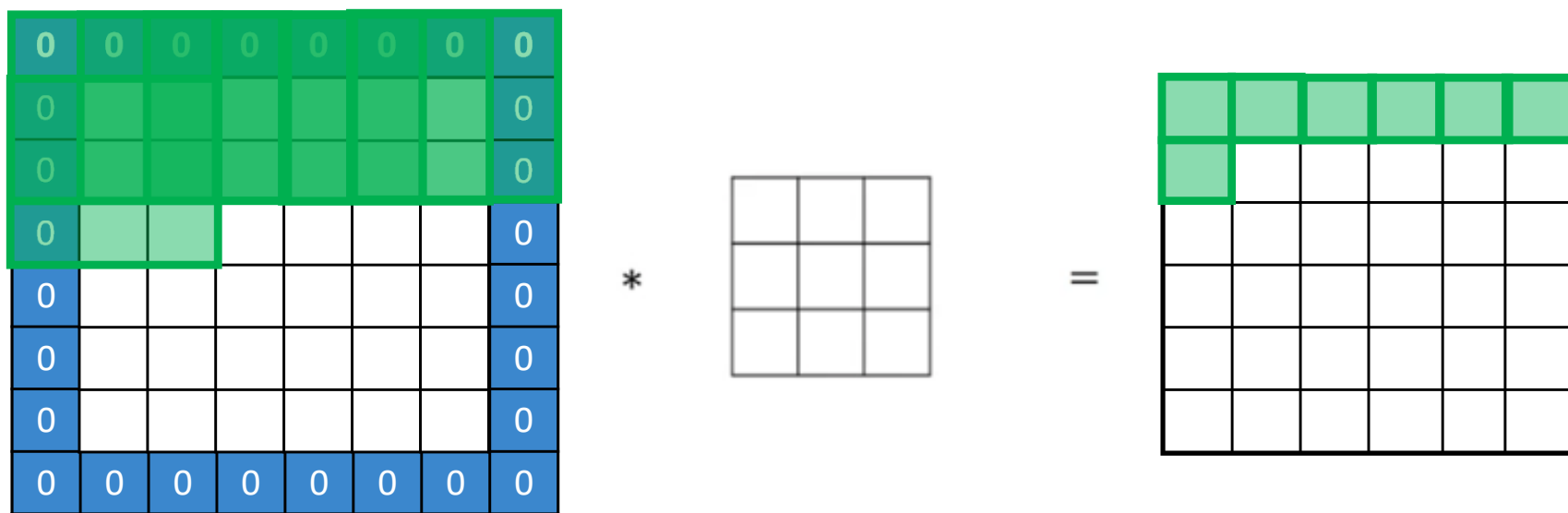
$$\begin{matrix} 6 \times 6 & \xrightarrow{p=1} & 8 \times 8 \\ h \times w & (h+2p) \times (w+2p) \end{matrix}$$

$$\begin{matrix} 3 \times 3 \\ k \times k \end{matrix}$$

$$4 \times 4 \longrightarrow ? \times ?$$

■ 填充 (Padding)

- 填充是构建卷积层的一个基本操作
- 在空间维度上，对卷积层输入的四周各填充 p 个像素



空间
维度

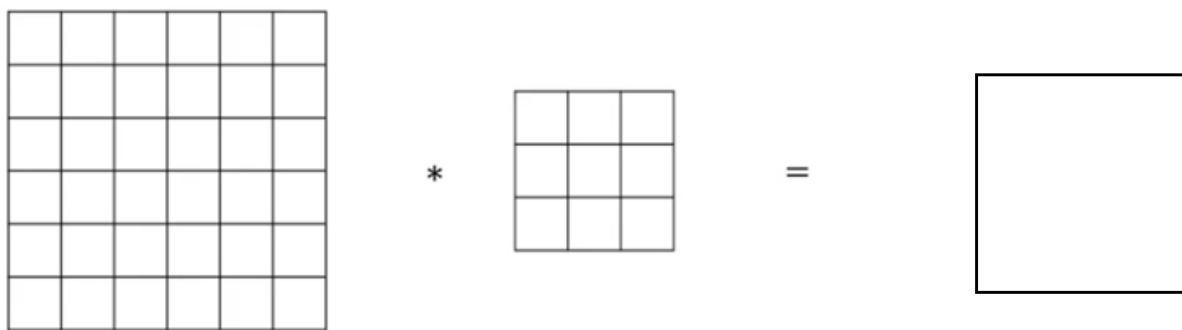
$$\begin{matrix} 6 \times 6 & \xrightarrow{p=1} & 8 \times 8 \\ h \times w & & (h+2p) \times (w+2p) \end{matrix}$$

$$\begin{matrix} 3 \times 3 \\ k \times k \end{matrix}$$

$$\begin{matrix} 4 \times 4 & \longrightarrow & 6 \times 6 \\ (h+2p-k+1) \times (w+2p-k+1) \end{matrix}$$

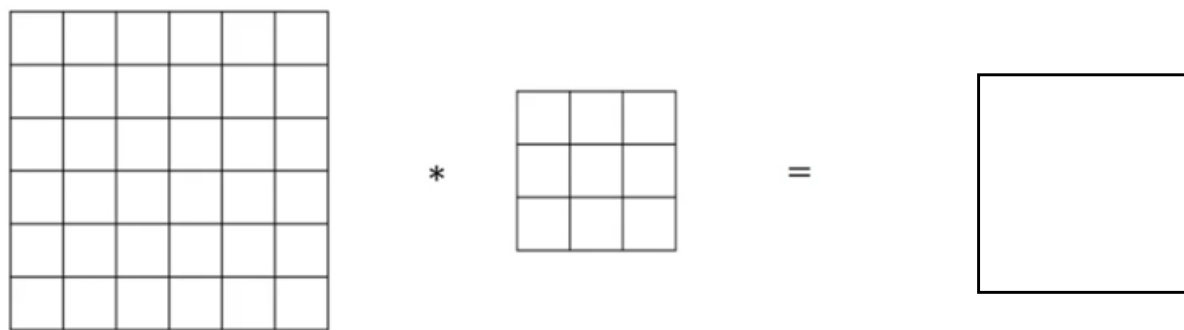
■ 填充 (Padding)

- 常用的两种padding方式: valid padding, same padding



■ 填充 (Padding)

➤ 常用的两种padding方式: **valid padding**, **same padding**

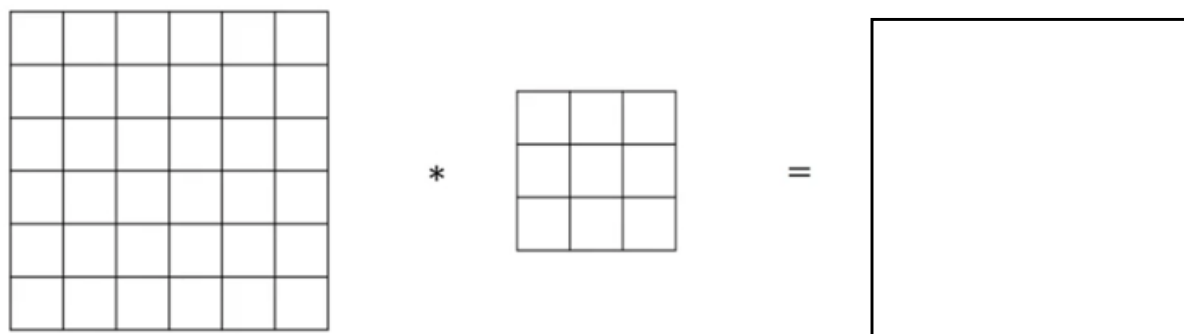


- **valid padding:** 对原始图像不做任何padding处理, 即 $p = 0$ 。此时, 在空间维度上, 卷积的输出小于输入

空间维度	$h \times w$	$k \times k$	$(h-k+1) \times (w-k+1)$
	6×6	3×3	4×4

■ 填充 (Padding)

- 常用的两种填充: **valid padding**, **same padding**



- **same padding**: 对原始图像进行填充处理 ($p > 0$), 并使得卷积后, 卷积的输出在空间维度上与输入保持一致。

空间维度	$(h+2p) \times (w+2p)$	$k \times k$	$(h+2p-k+1) \times (w+2p-k+1)$
	$(6+2) \times (6+2)$	3×3	6×6

各边均填充 p 个像素点

此时, $p = (k + 1)/2$

■ 步幅 (Stride)

- 卷积中的步幅是另一个构建神经网络基本操作
- 作用：成倍地缩小卷积输出的空间尺寸，且缩小倍数等于**Stride**的值

2	3	7	4	6	2	9
6	6	9	8	7	4	3
3	4	8	3	8	9	7
7	8	3	6	6	3	4
4	2	1	8	3	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

7×7

*

3	4	4
1	0	2
-1	0	3

3×3

■ 步幅 (Stride)

- 卷积中的步幅是另一个构建神经网络基本操作
- 作用：成倍地缩小卷积输出的空间尺寸，且缩小倍数等于Stride的值

2 ³	3 ⁴	7 ⁴	4	6	2	9
6 ¹	6 ⁰	9 ²	8	7	4	3
3 ⁻¹	4 ⁰	8 ³	3	8	9	7
7	8	3	6	6	3	4
4	2	1	8	3	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

7×7

*

3	4	4
1	0	2
-1	0	3

3×3

=

91		

3×3

- 假设Stride=2, 则在空间维度上，输出大小为输入的1/2

■ 步幅 (Stride)

- 卷积中的步幅是另一个构建神经网络基本操作
- 作用：成倍地缩小卷积输出的空间尺寸，且缩小倍数等于Stride的值

2	3	7 ³	4 ⁴	6 ⁴	2	9
6	6	9 ¹	8 ⁰	7 ²	4	3
3	4	8 ⁻¹	3 ⁰	8 ³	9	7
7	8	3	6	6	3	4
4	2	1	8	3	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

7×7

*

3	4	4
1	0	2
-1	0	3

3×3

=

91	100	

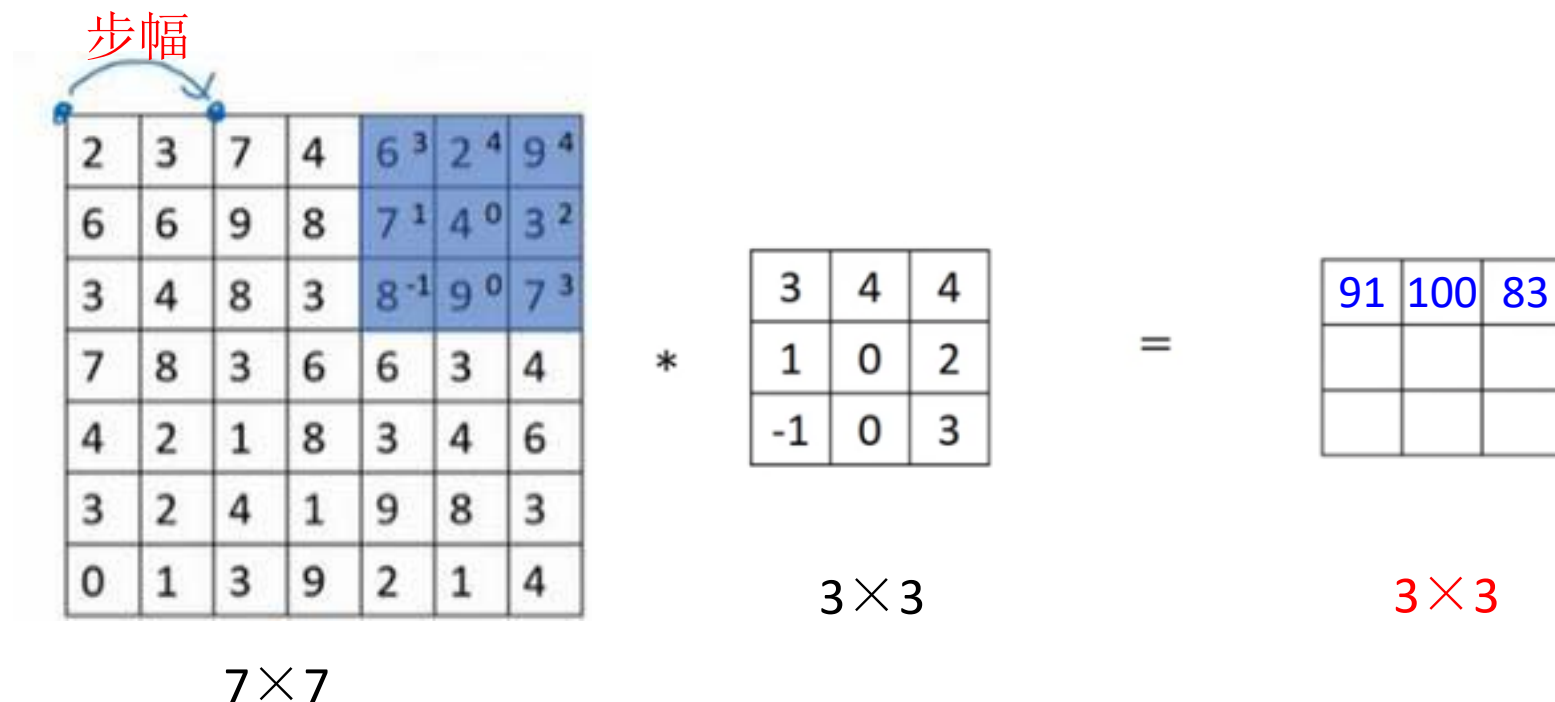
3×3

- 假设Stride=2, 则在空间维度上，输出大小为输入的1/2

■ 步幅 (Stride)

- 卷积中的步幅是另一个构建神经网络基本操作
- 作用：成倍地缩小卷积输出的空间尺寸，且缩小倍数等于Stride的值

步幅



2	3	7	4	6 ³	2 ⁴	9 ⁴
6	6	9	8	7 ¹	4 ⁰	3 ²
3	4	8	3	8 ⁻¹	9 ⁰	7 ³
7	8	3	6	6	3	4
4	2	1	8	3	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

7×7

*

3	4	4
1	0	2
-1	0	3

3×3

=

91	100	83

3×3

- 假设Stride=2, 则在空间维度上，输出大小为输入的1/2

■ 步幅 (Stride)

- 卷积中的步幅是另一个构建神经网络基本操作
- 作用：成倍地缩小卷积输出的空间尺寸，且缩小倍数等于Stride的值

步幅

步幅

2	3	7	4	6	2	9
6	6	9	8	7	4	3
3 ³	4 ⁴	8 ⁴	3	8	9	7
7 ¹	8 ⁰	3 ²	6	6	3	4
4 ⁻¹	2 ⁰	1 ³	8	3	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

7×7

*

3	4	4
1	0	2
-1	0	3

3×3

=

91	100	83
69		

3×3

- 假设Stride=2, 则在空间维度上，输出大小为输入的1/2

■ 步幅 (Stride)

- 卷积中的步幅是另一个构建神经网络基本操作
- 作用：成倍地缩小卷积输出的空间尺寸，且缩小倍数等于Stride的值

步幅

步幅

2	3	7	4	6	2	9
6	6	9	8	7	4	3
3	4	8 ¹	3 ⁴	8 ⁴	9	7
7	8	3 ¹	6 ⁰	6 ²	3	4
4	2	1 ¹	8 ⁰	3 ³	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

7×7

*

3	4	4
1	0	2
-1	0	3

3×3

=

91	100	83
69	91	

3×3

- 假设Stride=2, 则在空间维度上，输出大小为输入的1/2

■ 步幅 (Stride)

- 卷积中的步幅是另一个构建神经网络基本操作
- 作用：成倍地缩小卷积输出的空间尺寸，且缩小倍数等于Stride的值

步幅

步幅

2	3	7	4	6	2	9
6	6	9	8	7	4	3
3	4	8	3	8 ¹	9 ⁴	7 ⁴
7	8	3	6	6 ¹	3 ⁰	4 ²
4	2	1	8	3 ⁻¹	4 ⁰	6 ³
3	2	4	1	9	8	3
0	1	3	9	2	1	4

7×7

*

3	4	4
1	0	2
-1	0	3

3×3

=

91	100	83
69	91	127

3×3

- 假设Stride=2, 则在空间维度上，输出大小为输入的1/2

■ 步幅 (Stride)

- 卷积中的步幅是另一个构建神经网络基本操作
- 作用：成倍地缩小卷积输出的空间尺寸，且缩小倍数等于Stride的值

步幅

步幅

2	3	7	4	6	2	9
6	6	9	8	7	4	3
3	4	8	3	8	9	7
7	8	3	6	6	3	4
4	2	1	8	3	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

7×7

*

3	4	4
1	0	2
-1	0	3

3×3

=

91	100	83
69	91	127
44		

3×3

- 假设Stride=2, 则在空间维度上，输出大小为输入的1/2

■ 步幅 (Stride)

- 卷积中的步幅是另一个构建神经网络基本操作
- 作用：成倍地缩小卷积输出的空间尺寸，且缩小倍数等于Stride的值

步幅

步幅

2	3	7	4	6	2	9
6	6	9	8	7	4	3
3	4	8	3	8	9	7
7	8	3	6	6	3	4
4	2	1 ²	8 ⁴	3 ⁴	4	6
3	2	4 ¹	1 ⁰	9 ²	8	3
0	1	3 ¹	9 ⁰	2 ³	1	4

7×7

*

3	4	4
1	0	2
-1	0	3

3×3

=

91	100	83
69	91	127
44	72	

3×3

- 假设Stride=2, 则在空间维度上，输出大小为输入的1/2

■ 步幅 (Stride)

- 卷积中的步幅是另一个构建神经网络基本操作
- 作用：成倍地缩小卷积输出的空间尺寸，且缩小倍数等于Stride的值

步幅

步幅

2	3	7	4	6	2	9
6	6	9	8	7	4	3
3	4	8	3	8	9	7
7	8	3	6	6	3	4
4	2	1	8	3 ¹	4 ¹	6 ¹
3	2	4	1	9 ¹	8 ⁰	3 ²
0	1	3	9	2 ¹	1 ⁰	4 ¹

7×7

*

3	4	4
1	0	2
-1	0	3

3×3

=

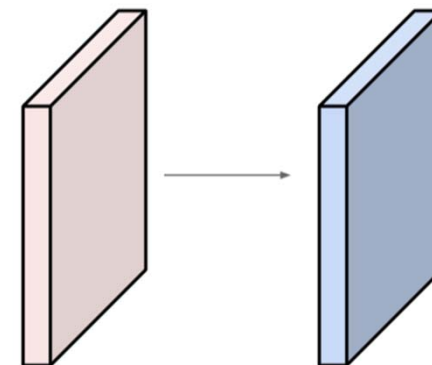
91	100	83
69	91	127
44	72	74

3×3

- 假设Stride=2, 则在空间维度上，输出大小为输入的1/2

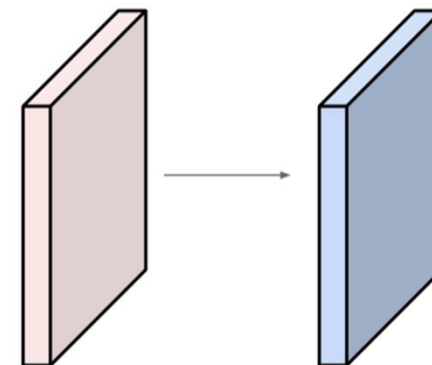
■ 测试

- 输入大小为 $32 \times 32 \times 3$
- 卷积层含有 10 个 $5 \times 5 \times 3$ 滤波器, $\text{stride}=1$, $\text{padding}=2$
- 问: 卷积的输出大小是多少?



■ 测试

- 输入大小为 $32 \times 32 \times 3$
- 卷积层含有 10 个 $5 \times 5 \times 3$ 滤波器, $\text{stride}=1$, $\text{padding}=2$
- 问: 卷积的输出大小是多少?

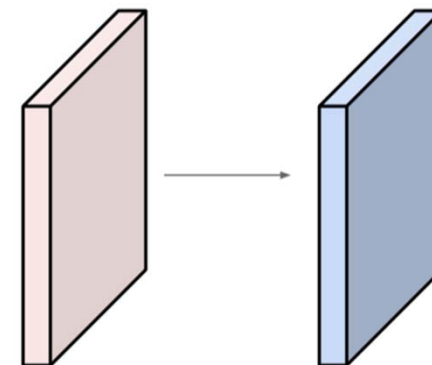


空间维度:
$$\frac{32 + 2 \times 2 - 5}{1} + 1 = 32$$

因此, 输出大小为: $32 \times 32 \times 10$

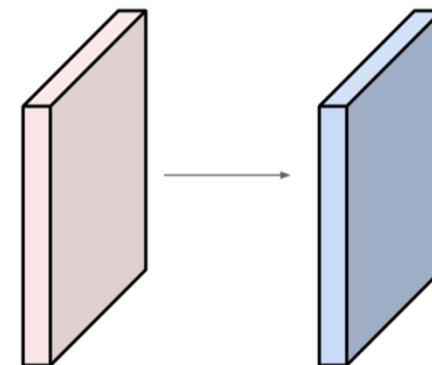
■ 测试

- 输入大小为 $32 \times 32 \times 3$
- 卷积层含有 10 个 $5 \times 5 \times 3$ 滤波器, $\text{stride}=1$, $\text{padding}=2$
- 问: 该层卷积的参数数量是多少?



■ 测试

- 输入大小为 $32 \times 32 \times 3$
- 卷积层含有 10 个 $5 \times 5 \times 3$ 滤波器, $\text{stride}=1$, $\text{padding}=2$
- 问: 该层卷积的参数数量是多少?



每个滤波器的参数量: $5 \times 5 \times 3 + 1 = 76$ (1 为偏置 bias)

因此, 10 个滤波器共: $76 \times 10 = 760$

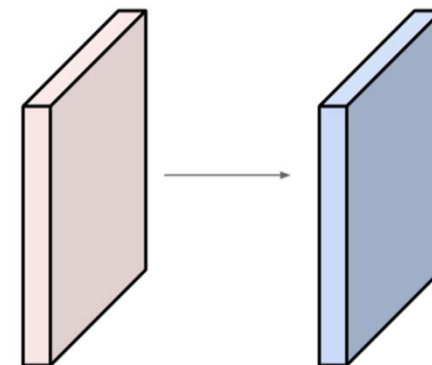
■ 卷积层小结

- 假设输入大小为 $W_1 \times H_1 \times C$
- 卷积层需要4个超参数：
 - 滤波器的个数 N 、卷积核大小 $K \times K$ 、
 - 步幅 S 、填充 P

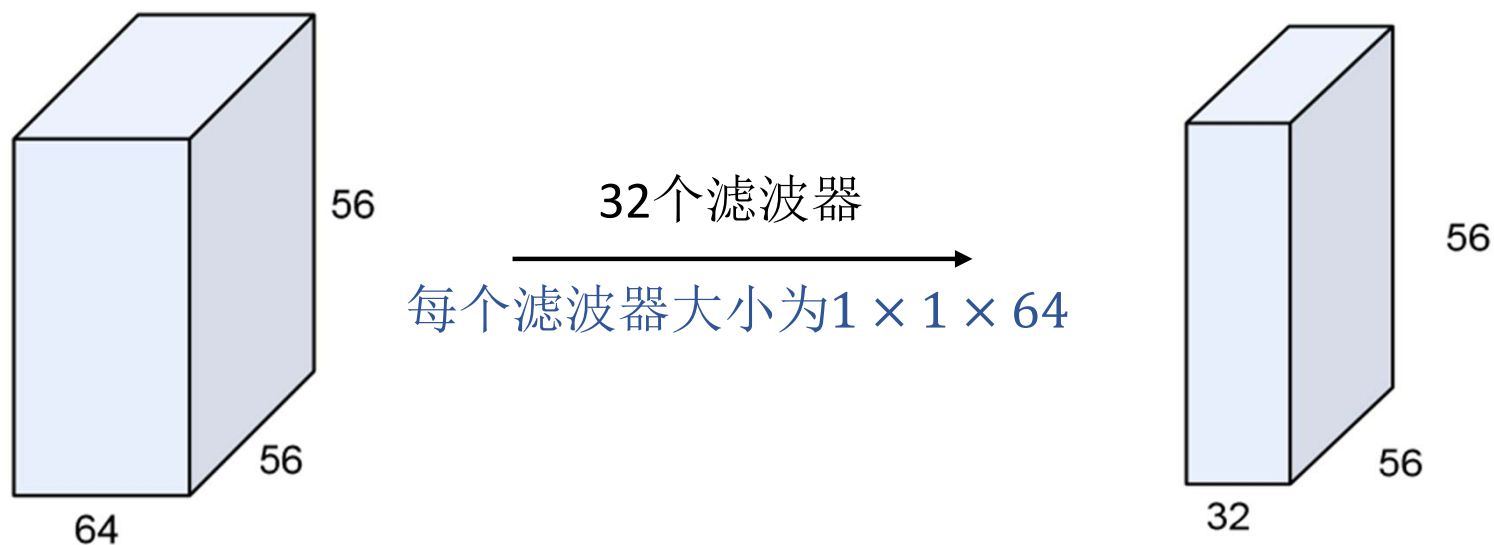
- 卷积的输出大小为 $W_2 \times H_2 \times N$ ，其中

$$W_2 = \frac{W_1 - K + 2P}{S} + 1, \quad H_2 = \frac{H_1 - K + 2P}{S} + 1$$

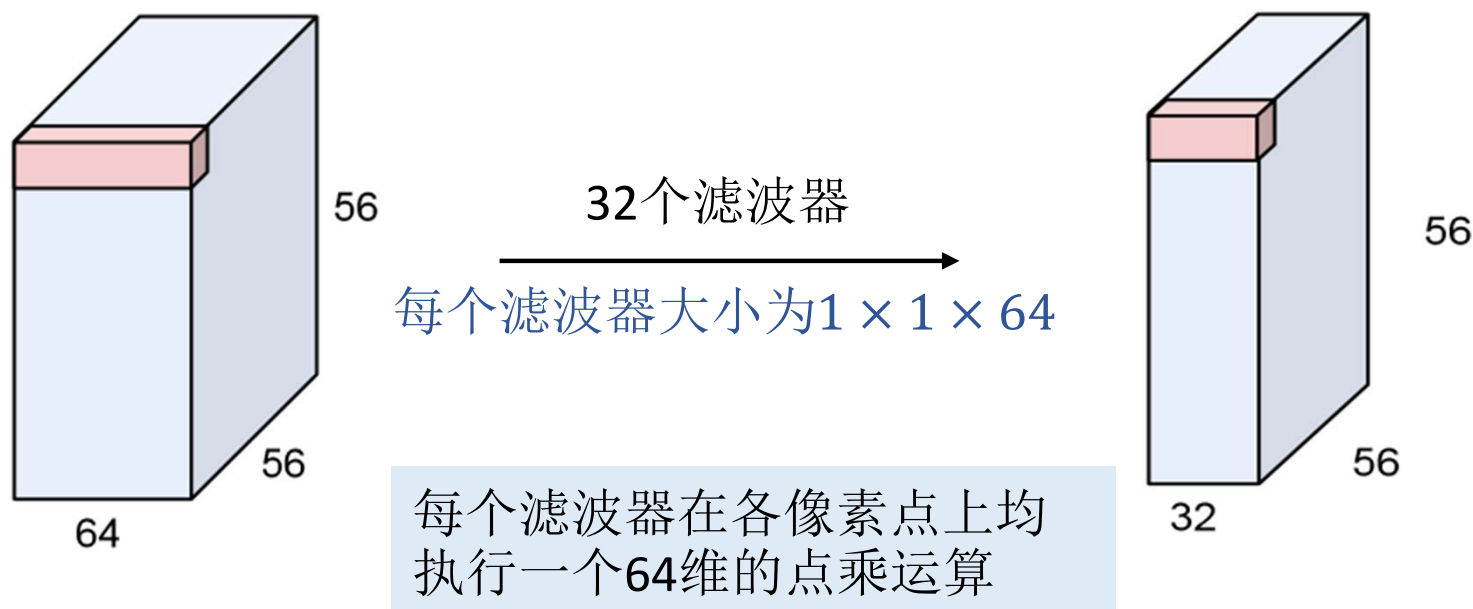
- 该卷积层的参数量为 $(K^2 \times C + 1) \times N$



- 值得一提的是，实际应用中， 1×1 的卷积滤波器经常被用到

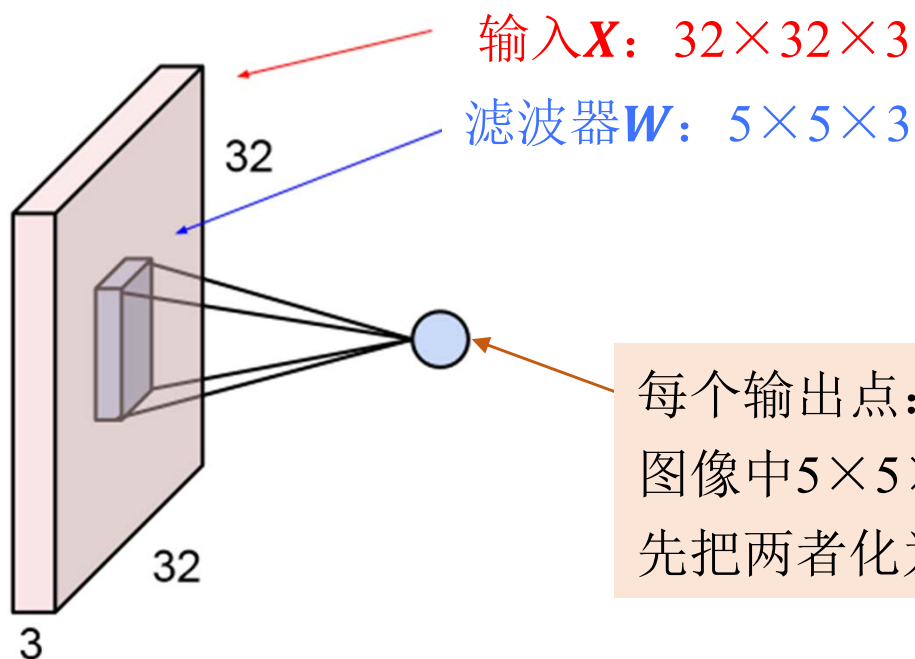


- 值得一提的是，实际应用中， 1×1 的卷积滤波器经常被用到

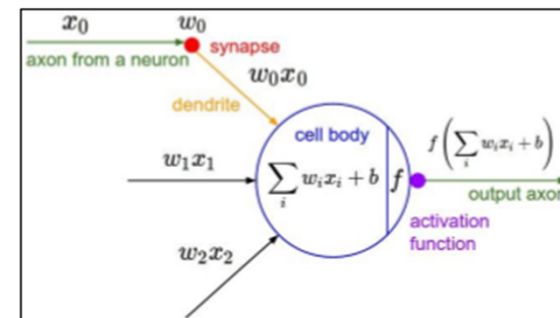


- 1×1 卷积通常用于特征降维，即在通道数方向上进行压缩
- 是减少网络计算量和参数的重要方式

➤ 神经元的感受野

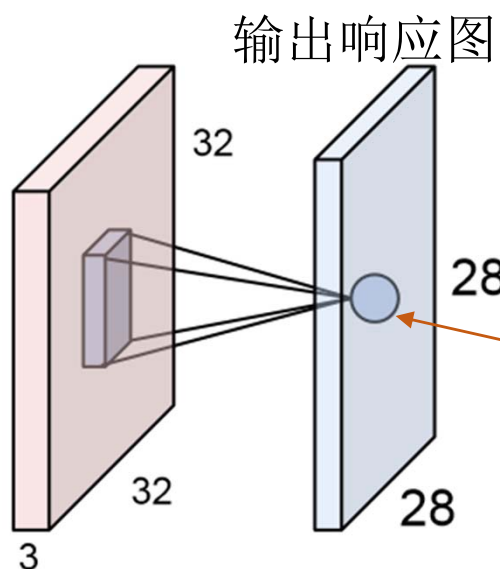


每个输出点：
图像中 $5 \times 5 \times 3$ 大小的单元与滤波器的内积，即：
先把两者化为 $5 \times 5 \times 3 = 75$ 维的向量，并计算 $\mathbf{w}^T \mathbf{x} + b$



反映了各神经元的局部连接

➤ 神经元的感受野



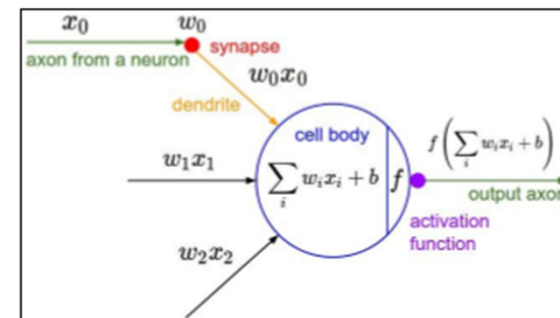
- 各点均与输入的一个 $5 \times 5 \times 3$ 局部区域相连
- 所有点均共享同一个滤波器（参数共享）

‘ 5×5 ’的卷积核 → 每个神经元 ‘ 5×5 ’的感受野

输入大小: $32 \times 32 \times 3$

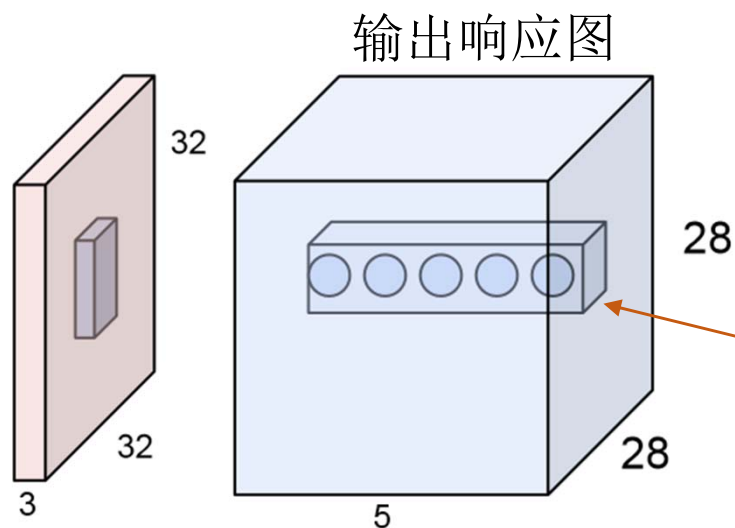
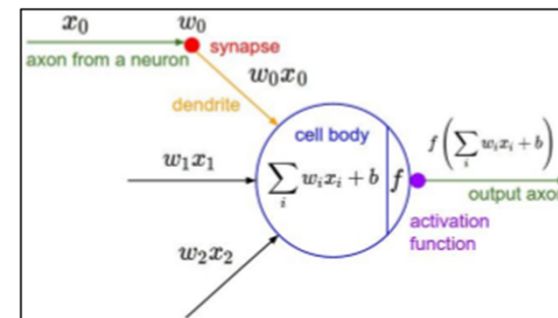
滤波器大小: $5 \times 5 \times 3$

输出大小: $28 \times 28 \times 1$



➤ 神经元的感受野

- 假设卷积层含有5个滤波器



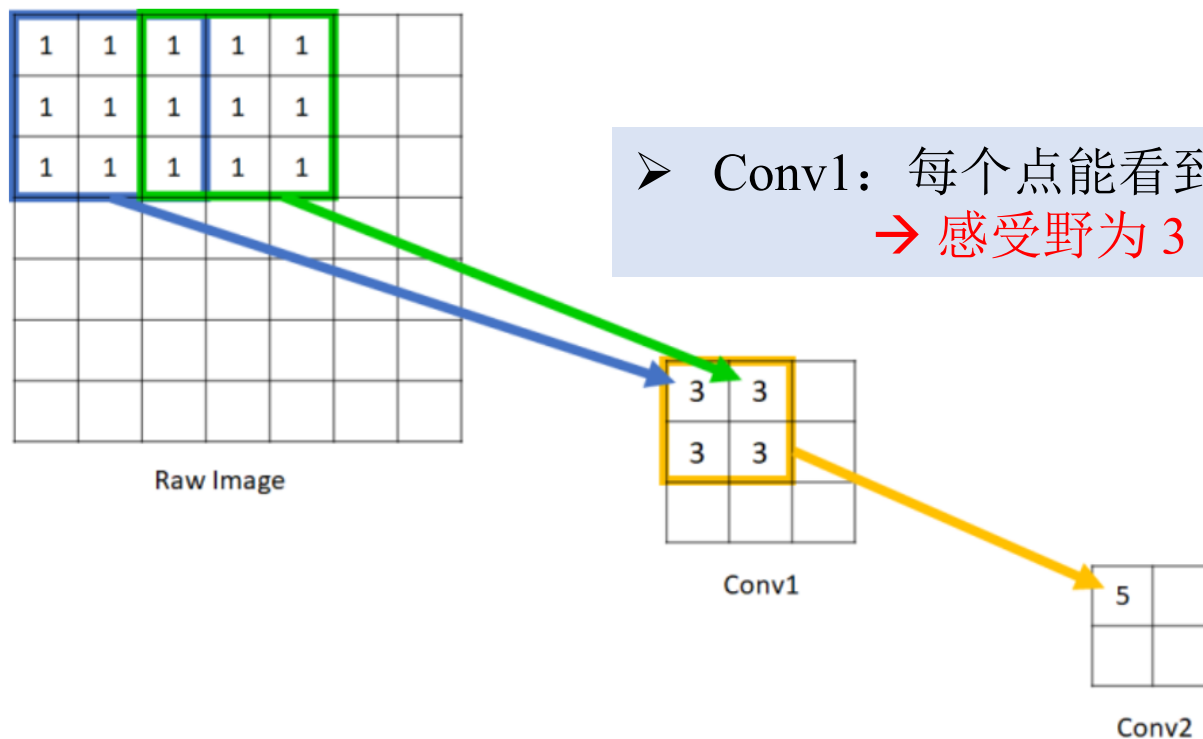
这5个不同的神经元同时看到输入的每一块 $5 \times 5 \times 3$ 局部区域

输入大小: $32 \times 32 \times 3$

滤波器大小: $5 \times 5 \times 3$

输出大小: $28 \times 28 \times 5$

➤ 神经元的感受野

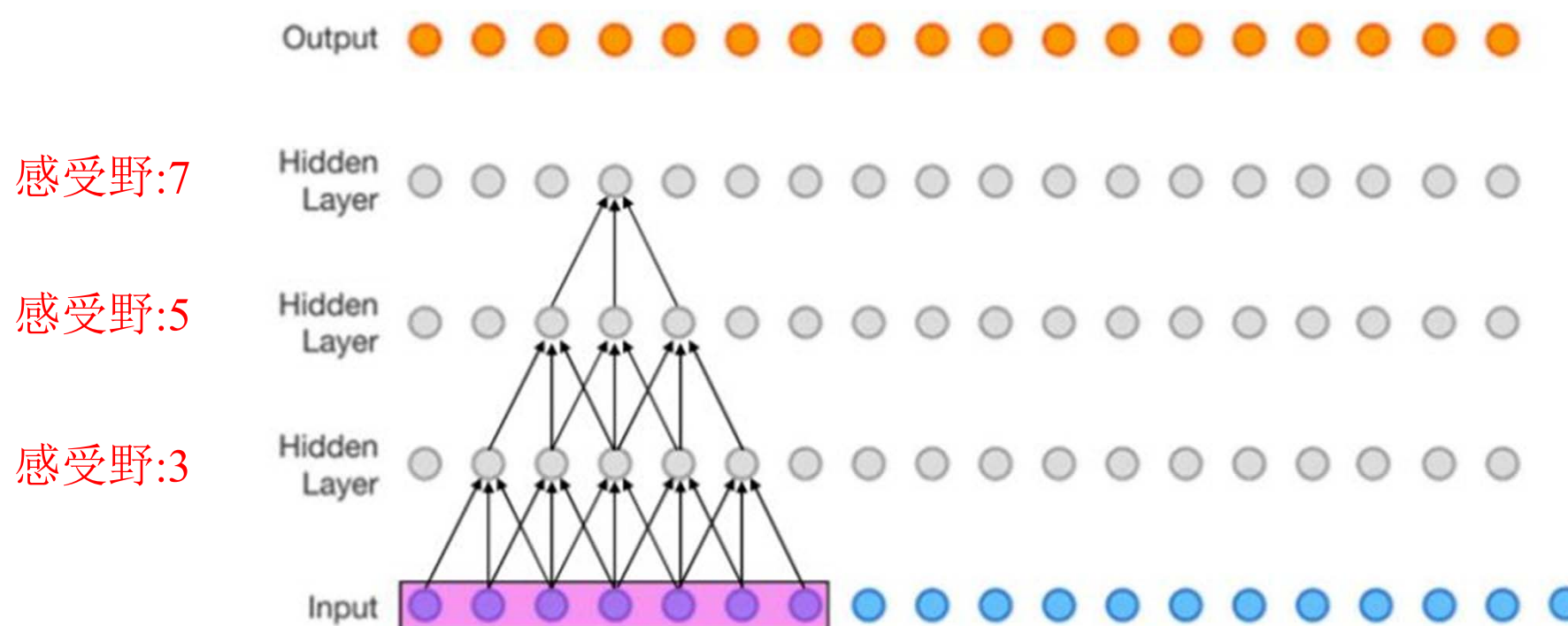


➤ Conv1: 每个点能看到的原始图像范围为 3×3 。
→ 感受野为 3

➤ Conv2: 每个单元均是由 2×2 的Conv1构成。回溯至原图像，实际可看到原始图像的范围为 5×5 。 → 感受野为 5

➤ 神经元的感受野

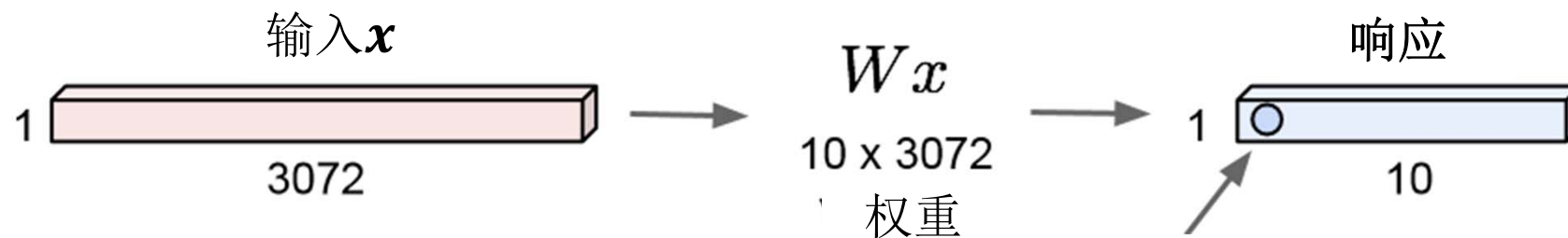
- 卷积层数越深，相应神经元的感受野越大



➤ 神经元的感受野

- 回顾：全连接层

$32 \times 32 \times 3$ 的输入 $\rightarrow$ 拉伸至 3072×1 的向量



每个输出点：权重的每一列与输入的内积结果（3072维的点乘）

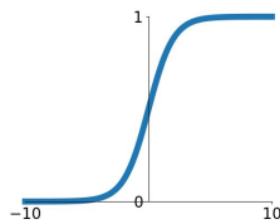
每个神经元均看到了输入的全部

■ 常见的几种激活函数

- 卷积处理过程可视为对各像素点赋予一个权值，该操作显然是线性的
- 加入非线性的激活函数，增强网络的非线性建模能力，对卷积层的输出特征进行过滤

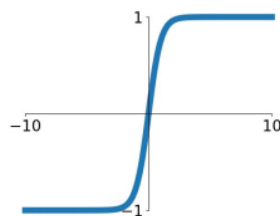
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



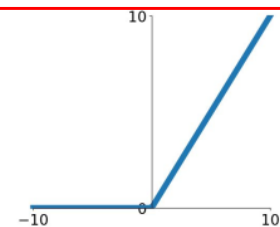
tanh

$$\tanh(x)$$



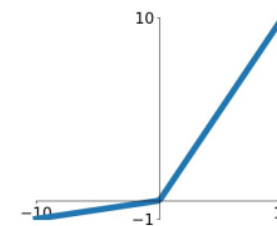
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

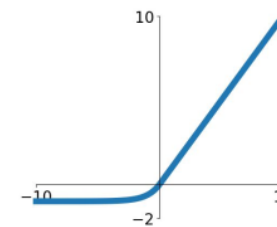


Maxout

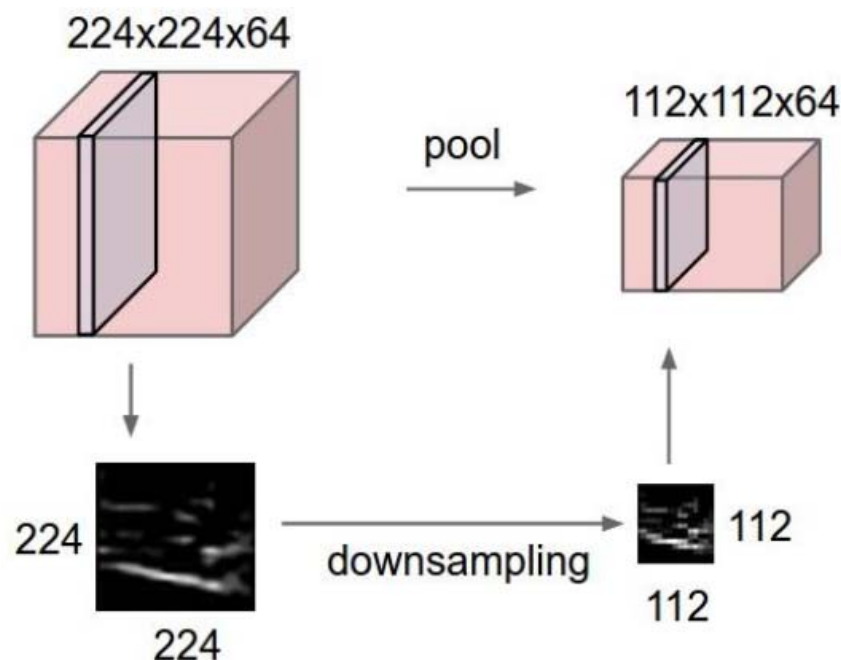
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

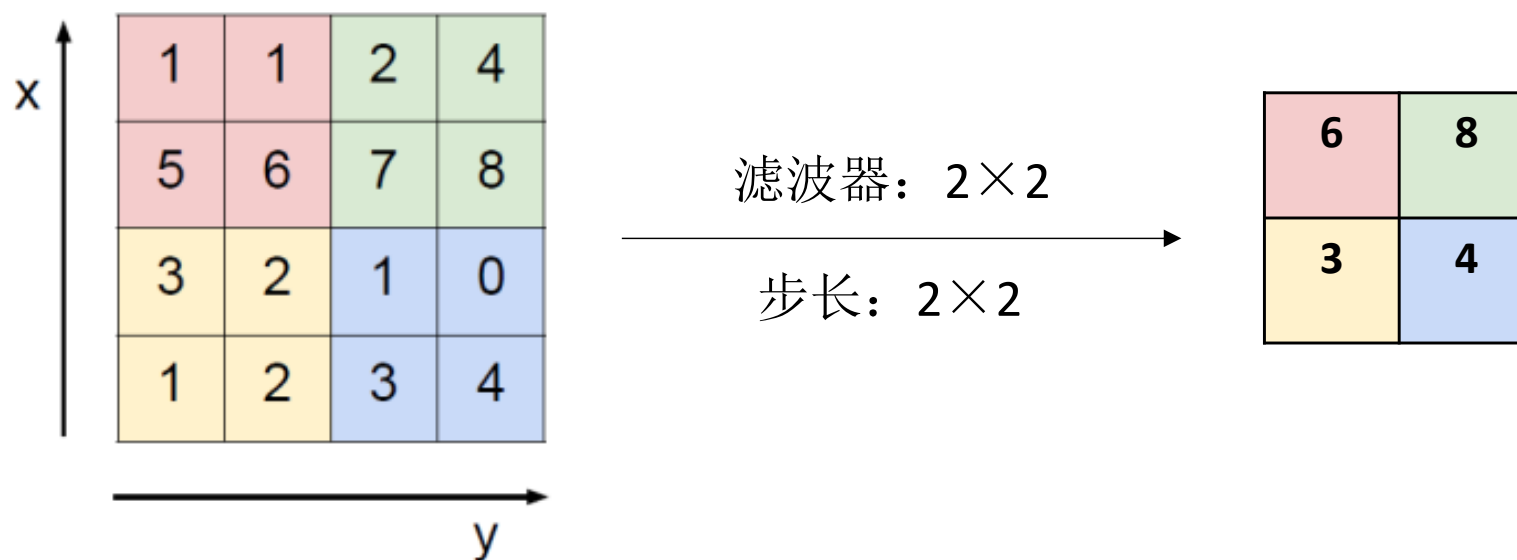


- 池化层 (Pooling Layer) 也叫子采样层 (Subsampling Layer)，其作用是进行特征选择，降低特征数量，从而减少参数数量。
 - 使用某一位置的相邻区域的总体统计特性来代替网络在该位置的输出
 - 在保留有用信息的同时，减少特征图的空间大小
 - 增大网络的感受野，且进一步减少后续层的参数数量



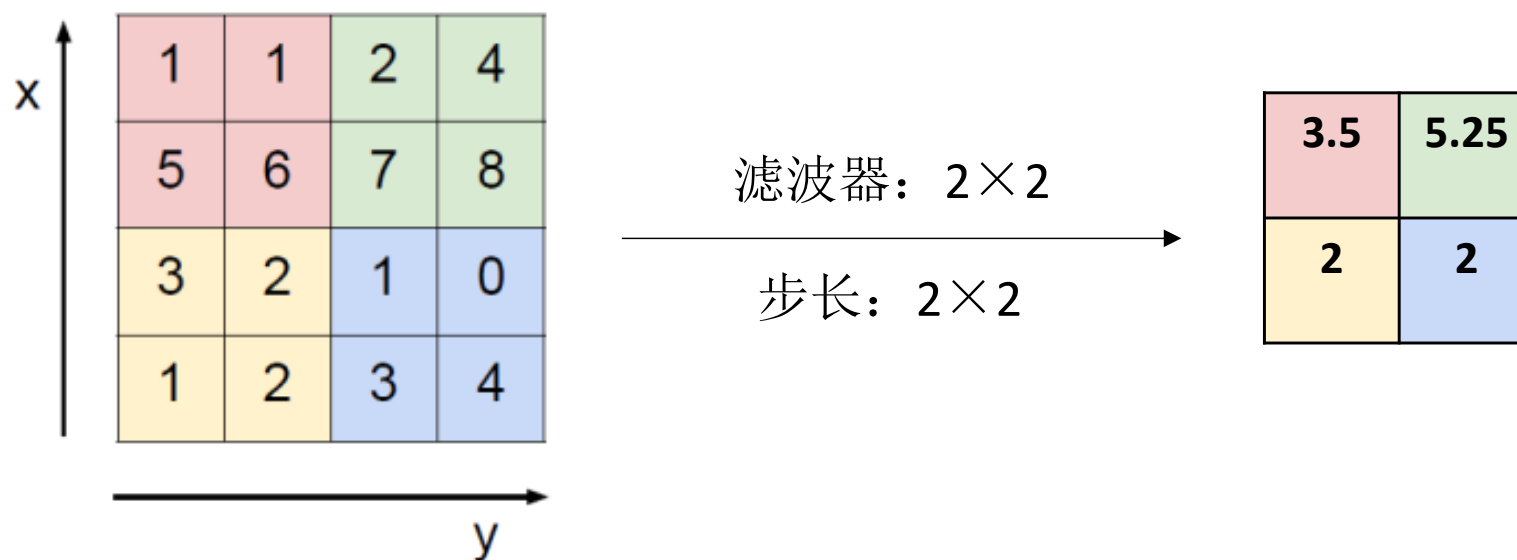
■ (1) 最大池化 (Max Pooling)

- 输出的每个元素都是其对应颜色区域中的最大值
- 涉及参数：滤波器大小、步长（通常与滤波器的大小相同）



■ (2) 平均池化 (Average Pooling)

- 输出的每个元素都是其对应颜色区域中的平均值
- 涉及参数：滤波器大小、步长（通常与滤波器的大小相同）

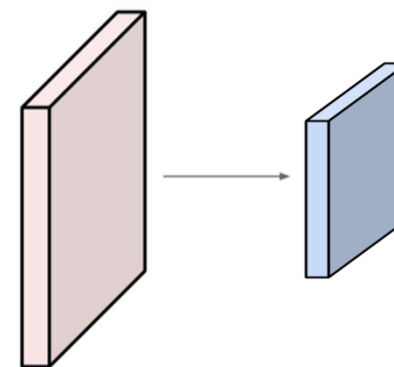


■ 池化层小结

- 假设输入大小为 $W_1 \times H_1 \times C$
- 池化层需要2个超参数：
 - 池化滤波器的大小 $K \times K$ 、步幅 S
- 池化后的输出大小为 $W_2 \times H_2 \times C$ ，其中

$$W_2 = \frac{W_1 - K}{S} + 1, \quad H_2 = \frac{H_1 - K}{S} + 1$$

- 池化层的参数数量为 0



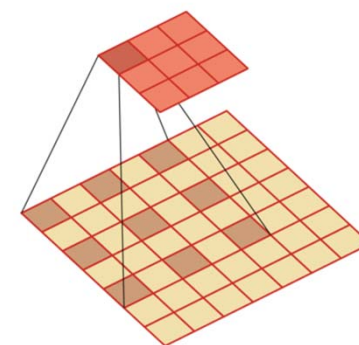
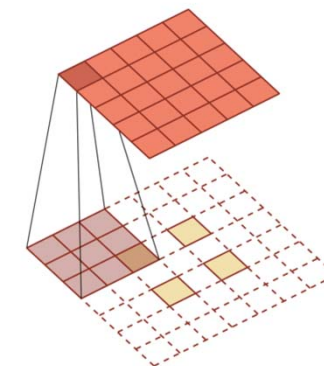
■ CNN中其他常用的网络层

➤ 批标准化层 (**Batch Normalization Layer**)

➤ 反卷积层 (**Deconvlutional Layer**)
将低维特征映射到高维特征的卷积操作

➤ 上池化层 (**Unpooling Layer**)

➤ 空洞卷积层 (**Dilated/Atrous Convolutional Layer**)
空洞卷积通过给卷积核插入“空洞”来变相地增加其大小
是一种不增加参数数量，同时增加输出单元感受野的一种方法

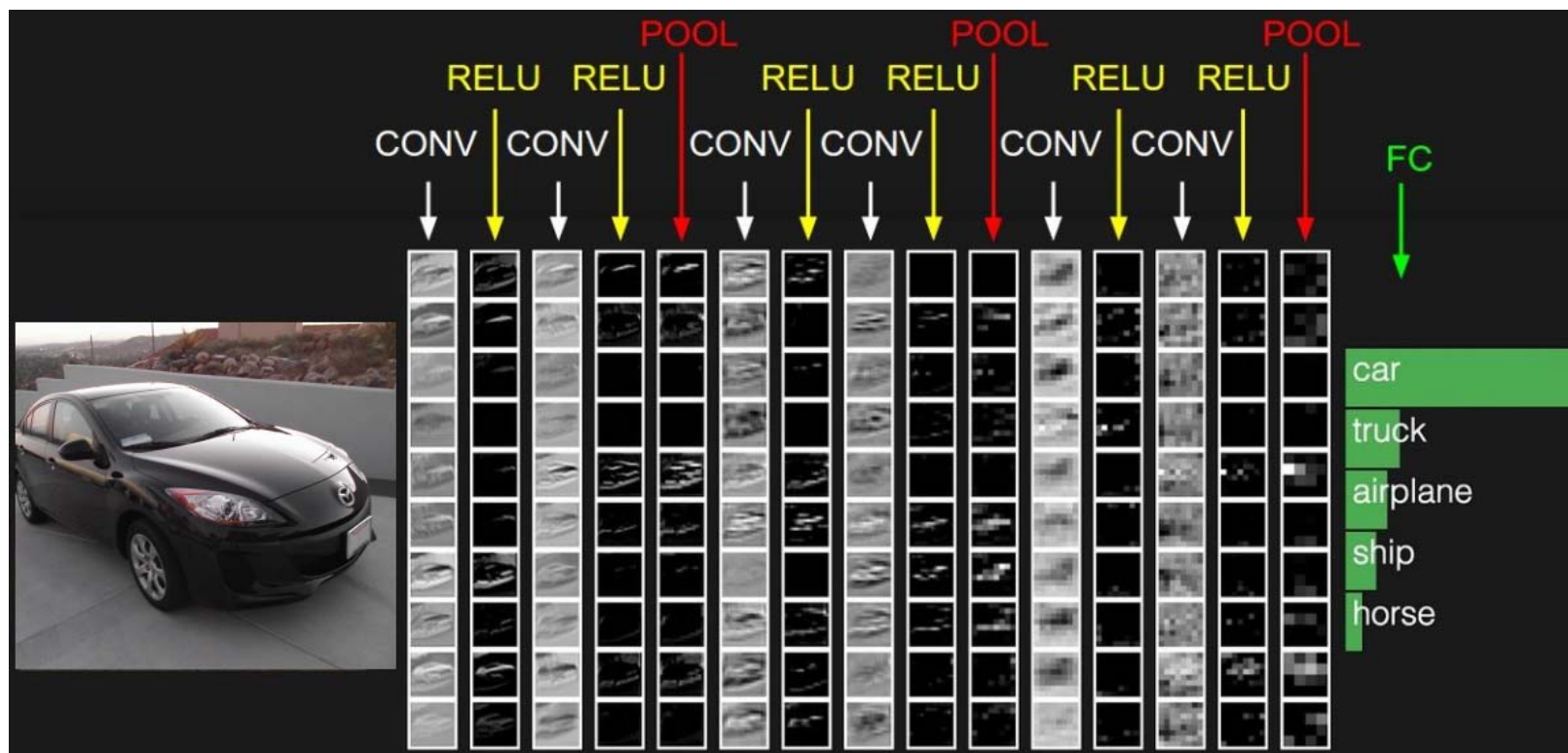


PART THREE

搭建卷积网络

CNN Construction

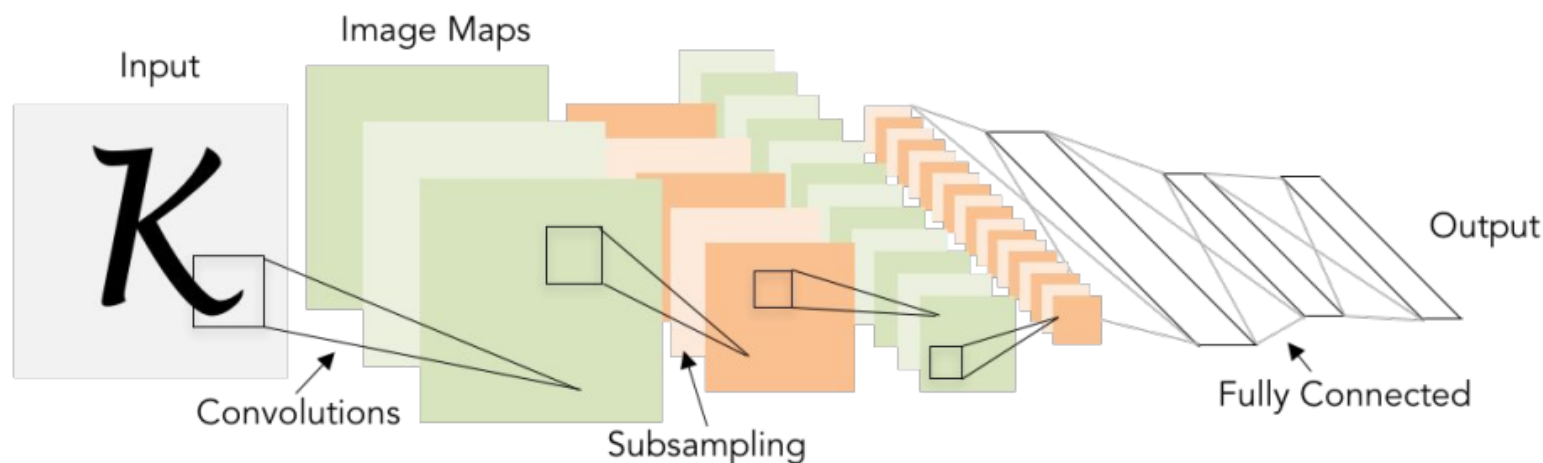
■ 搭建卷积神经网络



$$F^i = \text{ReLU}(F^{i-1} * W^i + b^i)$$

■ 以LeNet-5为例

- LeNet-5 是一个早期非常成功的CNN模型
- 基于 LeNet-5 的手写数字识别系统在 90 年代被美国很多银行使用，用来识别支票上面的手写数字。

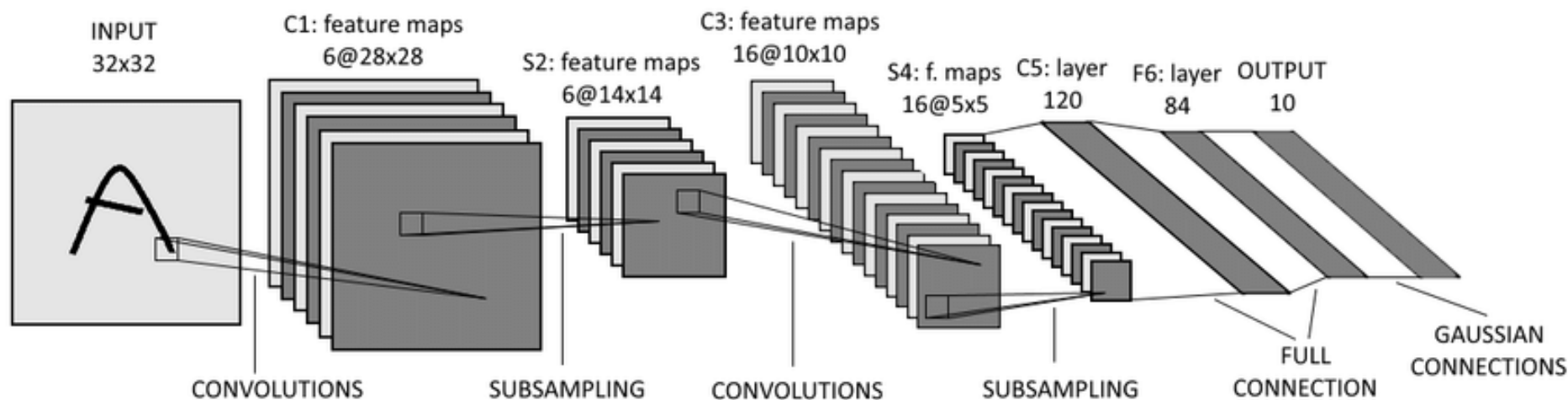


- Conv layers: 5×5 , stride 1
- Subsampling(Pooling) layers: 2×2 , stride=2
- i.e. architecture is [CONV-POOL-CONV-POOL-FC-FC]

■ 以LeNet-5为例

➤ INPUT: 输入层

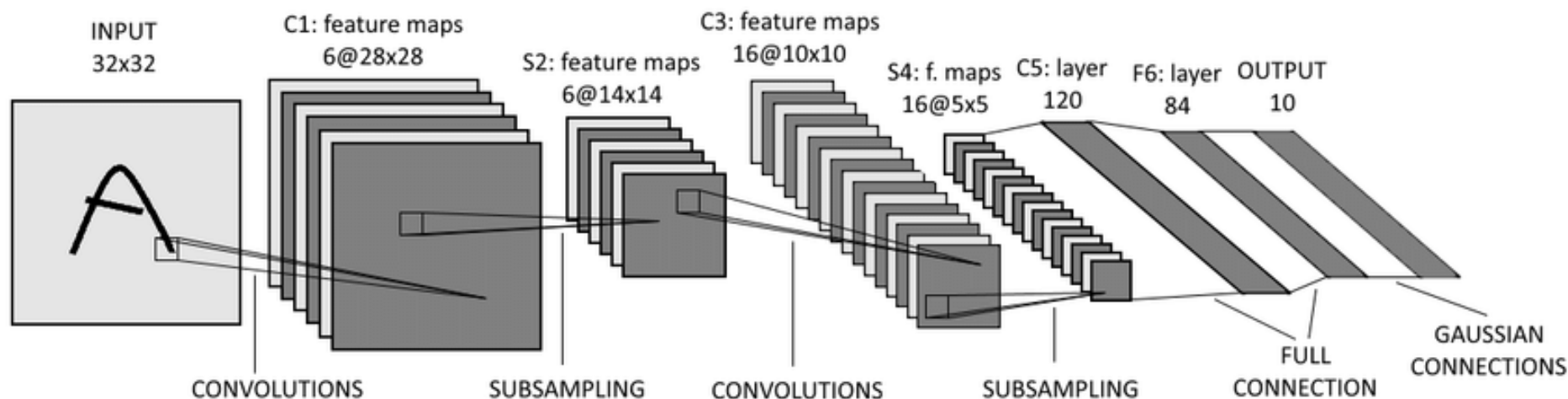
- LeNet-5的默认输入数据必须是维度为 $32 \times 32 \times 1$ 的灰度图像



■ 以LeNet-5为例

➤ C1层—第一个卷积层

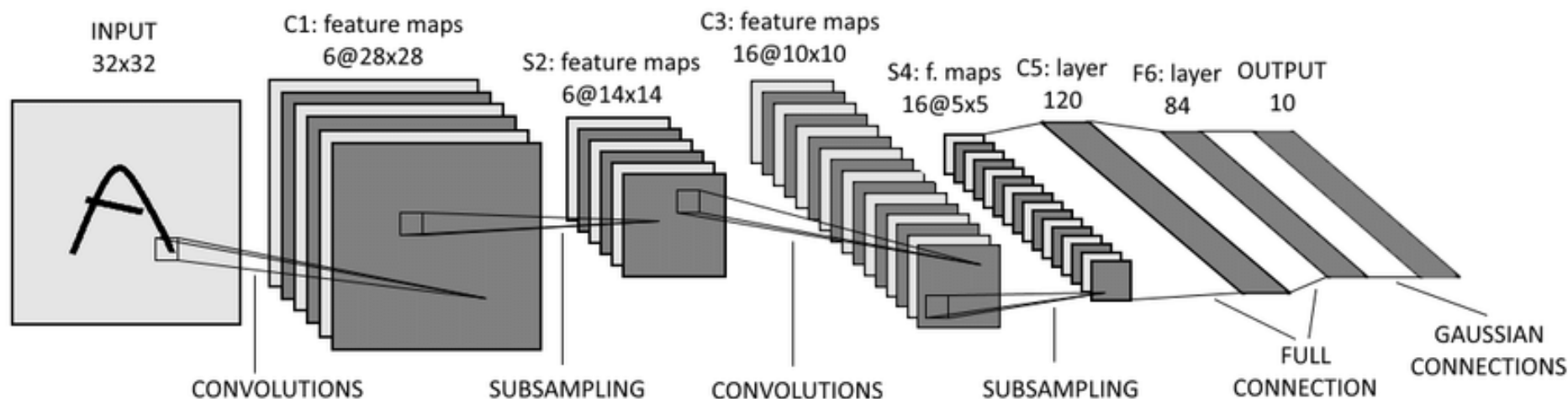
- 卷积核大小 $5 \times 5 \times 1$ ，步长1，不使用padding
- 共6个卷积核，输出的feature maps为6通道
- 输入大小为 $32 \times 32 \times 1$ ，输出为 $28 \times 28 \times 6$
- 参数共享：每个feature map内共享参数，该层参数量： $(5 \times 5 + 1) \times 6 = 156$



■ 以LeNet-5为例

➤ S2层—第一个Pooling层（下采样层）

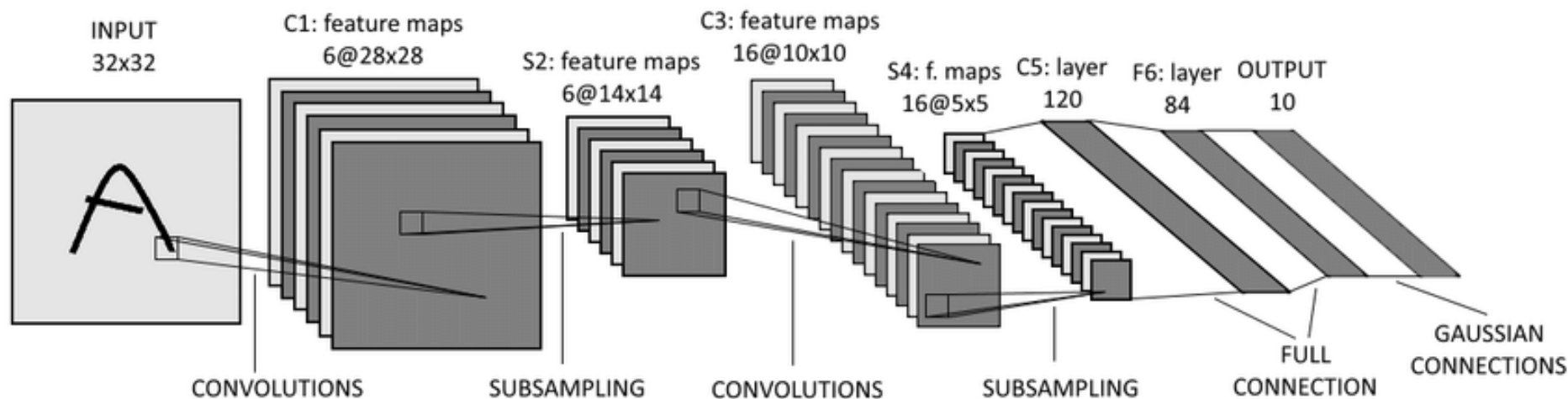
- 利用Max Pooling，缩减输入特征图 (即C1的输出) 的空间大小
- Max Pooling的滑动窗口大小 2×2 ，步长为2
- 输入特征图 $28 \times 28 \times 6$ ，输出特征图 $14 \times 14 \times 6$ （ $14 = (28 - 2) / 2 + 1$ ）
- 该层参数量为0



■ 以LeNet-5为例

➤ C3层—第二个卷积层

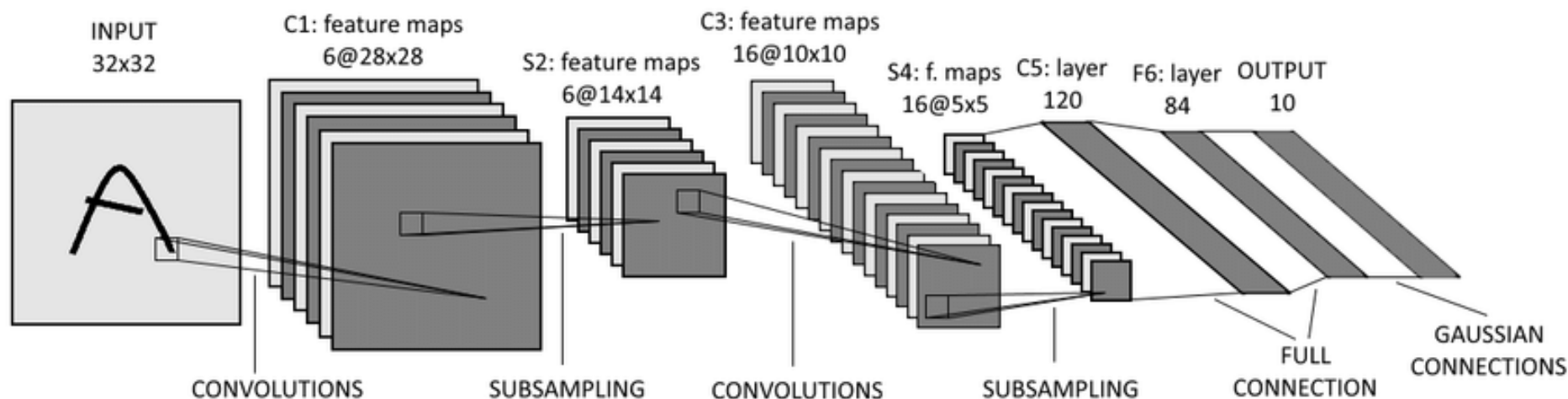
- 卷积核大小 $5 \times 5 \times 6$ ，步长为1，不使用padding
- 共16个卷积核，输出的feature maps为16通道
- 输入大小为 $14 \times 14 \times 6$ ，输出为 $10 \times 10 \times 16$
- 参数共享：每个feature map内共享参数，参数总数为： $(5 \times 5 + 1) \times 16$



■ 以LeNet-5为例

➤ S4层—第二个Pooling层（下采样层）

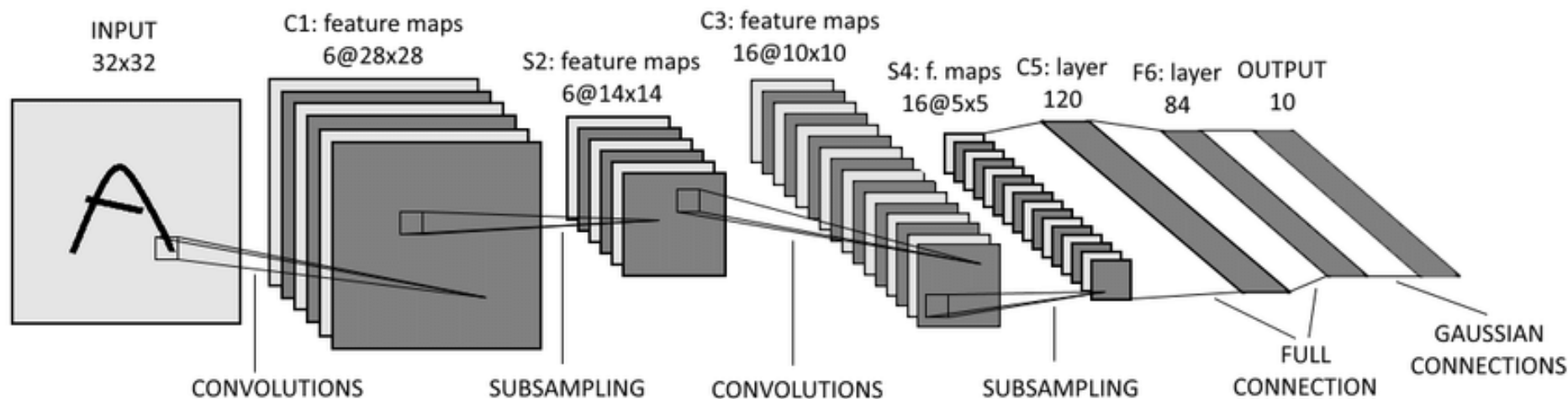
- 利用Max Pooling，进一步缩减输入特征图的空间大小
- Max Pooling的滑动窗口大小 2×2 ，步长2
- 输入特征图 $10 \times 10 \times 16$ ，输出特征图 $5 \times 5 \times 16$ （ $5 = (10 - 2) / 2 + 1$ ）
- 该层参数量为 0



■ 以LeNet-5为例

➤ C5层—第三个卷积层

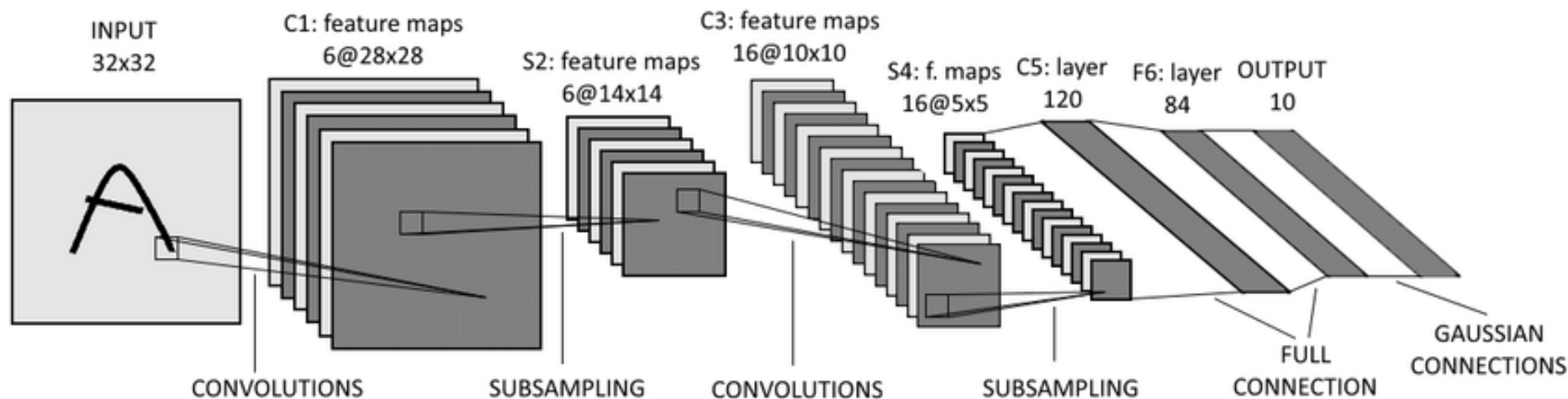
- 卷积核大小 $5 \times 5 \times 16$ ，步长为1，不使用padding
- 共120个卷积核，输出的feature maps为120通道
- 输入大小为 $5 \times 5 \times 16$ ，输出为 $1 \times 1 \times 120$
- 参数共享：每个feature map内共享参数，参数总数为： $(5 \times 5 + 1) \times 120$



■ 以LeNet-5为例

➤ F6层—第一个全连接层

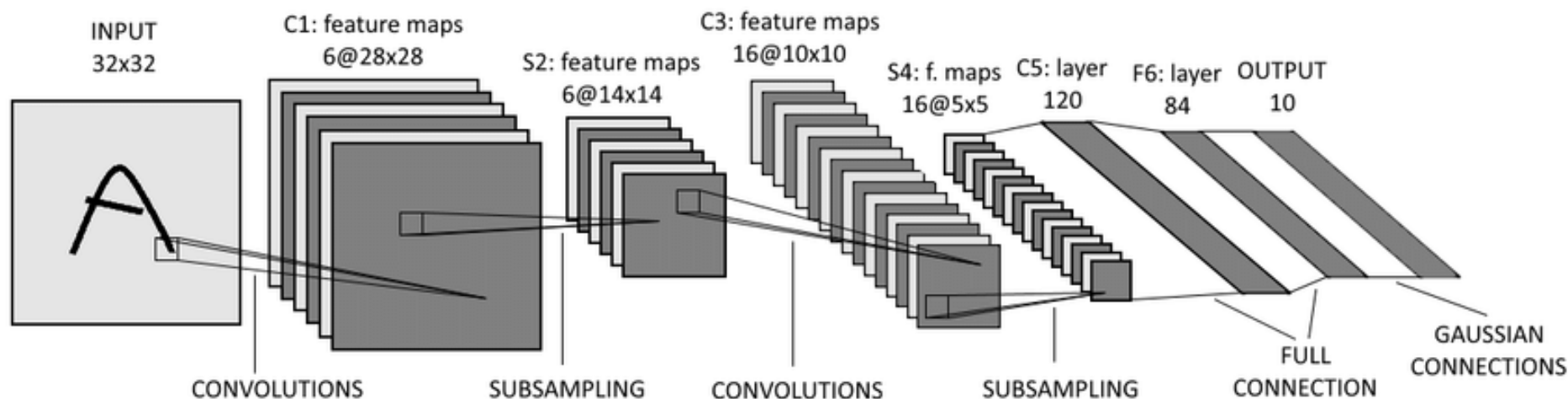
- 对输入特征进行维度压缩，输出 84×1 维的特征向量
- 将C5的输出 ($1 \times 1 \times 120$ 三维特征) 转换为 120×1 的特征向量作为输入
- 计算输入向量和权重向量间的乘积，再加上一个偏置
- 输入特征 120×1 ， 输出特征 84×1
- 该层可训练参数为： $(120+1) \times 84=10164$



■ 以LeNet-5为例

➤ OUTPUT层—输出层（第二个全连接层+Softmax）

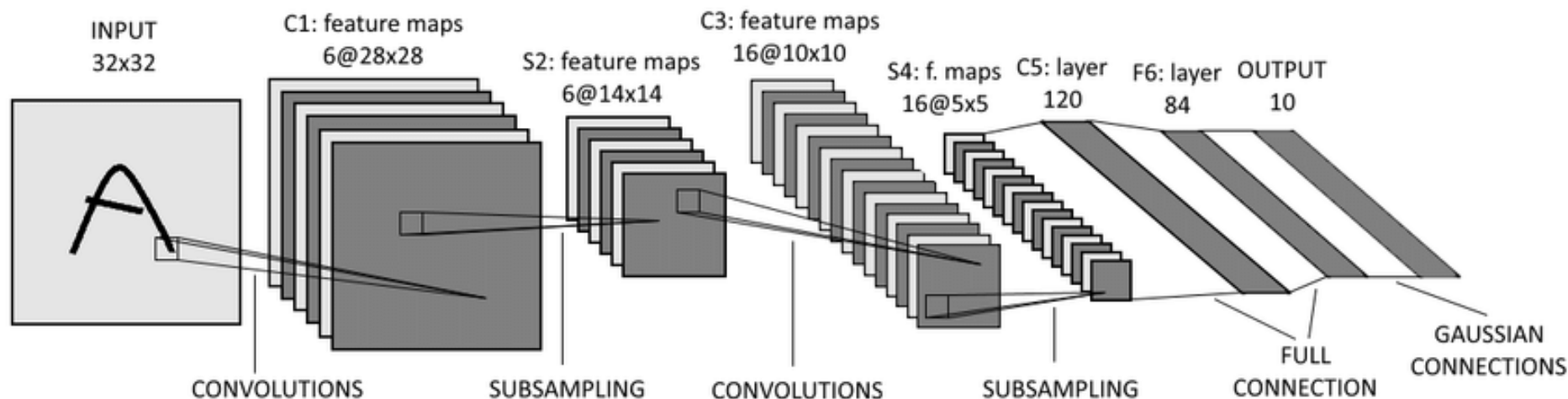
- 解决分类问题：判断输入图像中手写字体的类别，输出结果为输入图像对应10个类别的可能性值
- 利用全连接层，将输入的 84×1 维特征压缩成为 10×1 维度的数据
- 将该 10×1 维数据输入到Softmax激活函数中，预测出输入图像所对应的10个类别的可能性值
- 该层可训练参数为： $(84+1) \times 10=850$



■ 以LeNet-5为例

➤ LeNet-5网络结构与现在的网络的一些区别

- LeNet-5使用的非线性函数为Sigmoid，现在常使用ReLU/Leaky ReLU
- LeNet-5在池化层之后引入非线性，现在通常在卷积层后通过激活函数获取非线性，在池化层后不再引入非线性
- LeNet-5最后一步是Gaussian Connections，采用了RBF函数（即径向欧氏距离函数），计算输入向量和参数向量之间的欧氏距离。目前已经被Softmax取代



PART FOUR

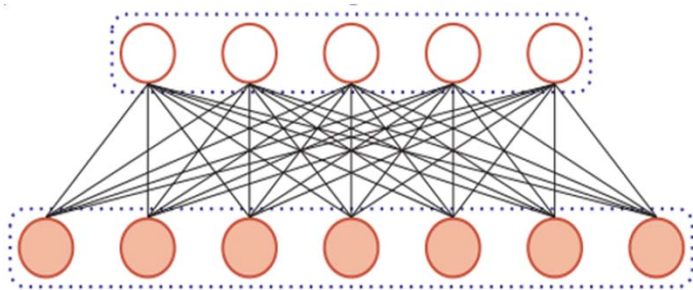
小结和资源

Take Home Message and Resource

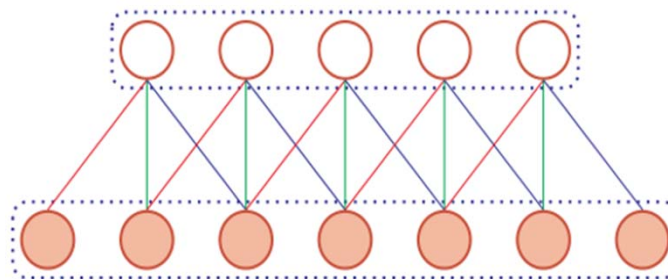
■ 卷积神经网络 (CNN)

- **局部连接**：每个神经元不再和上一层的所有神经元相连，而只和一小部分神经元相连。这样就减少了很多参数
- **权值共享**：一组连接可以共享同一个权重，而不是每个连接有一个不同的权重，这样又减少了很多参数
- **下采样(Pooling层)**：缩减特征图尺寸，增大网络感受野，且进一步减少后续层的参数

广泛应用于计算机视觉与图像处理领域！



全连接层



卷积层

■ 基础组件与基础网络

➤ 卷积层

- 参数：滤波器数量、卷积核大小、卷积步长 (Stride)、填充 (Padding)

➤ 激活层 (ReLU、Leaky ReLU较为常用)

➤ 池化层

- 最大池化、平均池化 (涉及池化窗口大小、步长)

■ 搭建卷积网络

- 以LeNet-5为例

➤ An MIT Press book

“Deep Learning” by Ian Goodfellow *et al.*

➤ Online Open Courses

Convolutional Neural Networks for Visual Recognition, Stanford.

➤ Paper

- [1] Krizhevsky *et al.*, ImageNet Classification with Deep Convolutional Neural Networks. NIPS2012.
- [2] Simonyan *et al.*, Very Deep Convolutional Networks for Large-Scale Image Recognition, ICLR2015.
- [3] Szegedy *et al.*, Going Deeper with Convolutions, CVPR2015.
- [4] He *et al.*, Deep Residual Learning for Image Recognition, CVPR2016.
- [5] Huang *et al.*, Densely Connected Convolutional Networks, CVPR2017.

